

Contents

Every line is a link. The page numbers are read back from the rendered book, never typed by hand.

How to use this manual	2
The exam at a glance	4
MODULE ONE · SYSTEMS, ANALYSTS AND THE LIFE CYCLE	7
Unit 1 · Systems Analysis and the Systems Analyst	8
Unit 2 · The System Development Life Cycle	11
Unit 3 · Methodologies, Feasibility and Approaches	14
MODULE TWO · REQUIREMENTS: DISCOVERY AND MODELLING	19
Unit 1 · Fact-Finding and Requirements	20
Unit 2 · Use Case Modelling	23
MODULE THREE · PROCESS AND DATA FLOW MODELLING	26
Unit 1 · Data Flow Diagrams	27
MODULE FOUR · DATA MODELLING WITH ENTITY-RELATIONSHIP DIAGRAMS	32
Unit 1 · The Entity-Relationship Model	33
MODULE FIVE · OBJECT-ORIENTED MODELLING WITH UML	38
Unit 1 · UML and Class Diagrams	39
Unit 2 · Object-Oriented Design Concepts	42
Unit 3 · Activity and Sequence Diagrams	43
MODULE SIX · DESIGN, ARCHITECTURE AND DELIVERY	46
Unit 1 · System Design and Architecture	47
Unit 2 · Project Management, Installation and Support	50
REFERENCE · THE NOTATION LEGEND AND MUST-MEMORISE LIST	53
The Notation Legend	54
The Must-Memorise List	55
WORKED PAPER 2025/2026	58
Section A: Questions One and Two	59
Section B: Questions Three to Six	63
WORKED PAPER 2024/2025	72
Section A: Questions One and Two	73
Section B: Questions Three to Six	75
MOCK EXAMINATION ONE	82
Question Paper	83
Answers	86
MOCK EXAMINATION TWO	89
Question Paper	90
Answers	93
MOCK EXAMINATION THREE	95
Question Paper	96
Answers	99

How to use this manual

Read this page once. It is short, and it decides whether the rest of the manual works for you.

TEACH

THE BARGAIN

This manual assumes you have never analysed a system, never drawn a use case or an entity-relationship diagram, and never heard of a class diagram. It carries you from there to a full mark in the examination. It is also meant to be **enough on its own**: you should never need a lecturer, a textbook or a website to follow it. Anything it uses, it teaches first, including every symbol of its own notation.

In exchange it asks one thing: **draw the diagrams yourself**. Reading a finished use case diagram or a solved data flow diagram creates a feeling of understanding that evaporates in the hall, where you face a blank page and a paragraph describing a business. Every diagram in this book was drawn by hand first from the words of a real question. Do the same, and the gap between your diagram and the one printed here is exactly the information you were missing.

The boxes

Every page is built from a small set of boxes. Each has one job, and the colour tells you which before you read a word.

TEACH

New ideas, explained from nothing. Read these properly.

● MUST MEMORISE

Emerald green means **learn this by heart**. Definitions the examiner asks for word for word, the phases of the life cycle, the four use case relationships, the crow's-foot symbols, the four object-oriented concepts. It is the only thing this colour is ever used for. If a page has green on it, there is something on that page you are expected to know cold.

WORKED

SOLVED RIGHT WHERE IT IS TAUGHT

A real question, solved in numbered steps, then a **now you try** at the end. Every worked box carries a small chip saying where the question comes from, so you know it is a question that was actually asked, not one invented to be easy:

PQ 25/26 Q4 a past paper (session and question number); **TEST 2** a class test; **TEXT** the course text's own exercise. Do the "now you try" before you move on.

⚠ TRAP

A mistake that is easy to make and costly in marks. Most are taken from the exact places the examiner sets a snare: a use case joined to another with the wrong arrow, a data flow drawn straight from an entity to a data store, a many-to-many relationship left with an attribute stranded on the line, an aggregation drawn where the question describes a composition.

RECALL

Questions with the answers printed underneath, below a dashed line. **Cover the answers.** Say your answer out loud, or write it, and only then look. Checking whether you knew something is not the same as knowing it, and the difference is the whole point of this box.

EXAM QUESTION

A question reproduced **as it appeared on the paper**, in a dark **As printed** bar, plain and uncoloured, so you meet it the way the examiner set it. A violet **Break it down** box then reads it in your shoes, and a teal **Answer** box works it in full.

Reading a figure

● MUST MEMORISE

THE NOTATION IS THE EXAM

More than half of this course is drawn, not written. An entity is a rectangle, a relationship a line with a verb on it, and the small marks at the ends of that line, the **crow's foot**, a circle, a single bar, carry the whole meaning of a business rule. A hollow diamond and a filled diamond look almost the same and mean opposite things. Get a symbol wrong and the answer is wrong, however good the prose around it. Every symbol used in this manual is defined the first time it appears, and the full legend sits in the Reference sheet. Where a figure is a diagram you must be able to draw, redraw it yourself until it comes out without looking.

One promise, and one label

TEACH

The promise. Every diagram, every solved question, and every past question quoted in this manual was worked out and transcribed independently, not copied from a model answer. The course's own materials and the papers themselves contain slips (you will meet a question that says "answer parts i to vii" and then prints eight, and a paper that misspells a word inside its own scenario), so nothing here is trusted just because a source printed it. Where a paper is wrong, this manual quotes it exactly and flags the error underneath, because in the hall you must answer the paper as printed.

The label. The course text covers most of what the examiner asks: the life cycle, requirements, use cases, data flow diagrams, entity-relationship diagrams and the UML diagrams are all in it. A few topics are examined but are not in the text, chiefly system architecture and delivery: the client-server tiers, hardware architecture, system acquisition, interface design and relational database design in Module Six. Where a topic is not in the course text, it carries a **NOT IN THE COURSE TEXT** label. It is still taught and solved in full, because it is worth real marks, and the label simply tells you that you will not find it if you go back to the text alone. This manual is where you find it. One more thing to know: the text draws entity-relationship diagrams in the **Chen** style (diamonds and ovals), while the exam asks for the **Information Engineering** style (the crow's foot). This manual teaches both and tells you which the paper wants.

BEFORE YOU START

The Exam at a Glance

What the paper looks like, how long you have, and which topics earn the marks. Learn this before you learn anything else: it tells you where to aim.

TEACH

THE SHAPE OF THE PAPER

The current paper runs for **three hours** and is worth 70 marks. It is split into two sections, and the instruction at the top is the first thing you read:

- ▶ **Section A** has two questions and you answer **one** of them. These are the broad, written questions: the life cycle and its phases, choosing a methodology, requirements and how you would gather them, feasibility, project planning. Few or no diagrams. This is where a candidate who has memorised the core lists scores fastest.
- ▶ **Section B** has four questions and you answer **any three**. These are the modelling questions, each built on a short business scenario: draw an entity-relationship diagram, a set of data flow diagrams, a use case diagram, a class diagram, an activity or sequence diagram, and explain the object-oriented concepts. Three of the four questions almost always ask for diagrams.

So you answer **four questions in all** (one from A, three from B) in three hours: about **forty minutes each**. That is generous, but only if you can draw the diagrams without hesitating, which is what the practice in this book buys you.

Across every past paper, the marks come from the same bank of topics. Learn this list; it is the whole course, and each item is a module in this manual.

- ▶ **The analyst and the life cycle** (Section A): what an analyst does, the five phases of the life cycle and their deliverables, the principles of a successful project.
- ▶ **Methodologies and feasibility** (Section A): predictive versus adaptive, waterfall, prototyping, rapid application development, the spiral, agile; when to use each; the feasibility checks.
- ▶ **Requirements** (Section A and B): elicitation techniques with their stakeholders and trade-offs; functional versus non-functional requirements.
- ▶ **Use case modelling** (Section B): actors, the four relationships, the diagram, the narrative.
- ▶ **Data flow diagrams** (Section B): the four elements, the context diagram, Process 0 and Level 1, and the mechanical errors.
- ▶ **Entity-relationship diagrams** (Section B): entities, keys, cardinality, the notation styles, and reading an existing diagram.
- ▶ **Class diagrams and object-oriented design** (Section B): classes with attributes and methods, relationships and multiplicities, and the concepts (encapsulation, inheritance, polymorphism, composition, aggregation).
- ▶ **Activity and sequence diagrams** (Section B): modelling a process flow and an interaction over time.
- ▶ **Design, architecture and delivery** (Section A and B): system architecture, acquisition strategy, installation, and support.

⚠ TRAP

READ THE RUBRIC

The number of questions and the split between sections has changed between sittings. Older papers under the previous code said "answer any three of five" in two hours with no sections at all; one year used "answer all of Section A and any two of Section B". The current paper is **one from Section A and three from Section B** in three hours. Never assume the rubric from a past paper you practised on. The first minute in the hall is spent reading the instruction at the top of the paper and counting how many questions from where. Answering the wrong number of questions, or all four from one section, throws away marks you had already earned.

A Section B question gives you a paragraph of business description and asks for several diagrams from it. The winning routine is always the same, and this manual drills it on every worked question:

- ▶ **Read for nouns and verbs first.** The nouns that matter become entities, classes or data stores; the verbs become relationships, processes or use cases. Underline them on the paper before you draw anything.
- ▶ **Draw in pencil-order:** boxes first, then the lines between them, then the symbols on the lines, then the labels. A diagram built in that order rarely has to be rubbed out.
- ▶ **Answer the verb in the instruction.** "Draw", "identify", "differentiate" and "explain" want different things. Drawing when it says explain, or explaining when it says draw, loses the marks even when what you wrote is correct.

Every worked question in this book opens with a **Break it down** box that does exactly this reading step, so that by the exam it is automatic.

MODULE ONE

Systems, Analysts and the Life Cycle

Before you can model a system you must know what a system is, what the person building it actually does, and the orderly path a project follows from a vague business complaint to a working, supported product. This module is the whole of Section A of the paper.



Systems Analysis and the Systems Analyst

By the end of this unit you can say what systems analysis and design is, describe what a systems analyst does and the skills the role needs, define an information system, and explain functional decomposition.

TEACH

WHAT THE COURSE IS ABOUT

Systems analysis is the process of understanding a business problem well enough to say exactly what a new system must do about it. **Systems design** is the process of deciding how the system will be built to do it. Analysis asks *what*; design asks *how*. Together they are the front half of building any information system, and they are what this course examines.

Analysis comes first and design follows, because you cannot sensibly decide how to build something until you know what it has to do. Analysis is a **discovery** process: its two watchwords are discovery and understanding. Design is a **decision** process: it turns the understanding into a blueprint of program structure, data, screens and hardware that a programmer can build from.

TEACH

WHO DOES IT

The person who does this work is the **systems analyst**. The single most important idea about the analyst, and one the examiner asks almost every year, is this: **a systems analyst is a business problem solver**. The analyst is not primarily a programmer. The analyst studies a business, finds where it is losing time or money or accuracy, works out whether technology can help, and if so specifies the system that will help. The solution to the problem is usually, but not always, an information system.

Because the analyst sits between the people with the problem and the people who will build the solution, the analyst is a **translator**: turning business objectives into functional requirements, user needs into system specifications, and technical jargon into terms an ordinary manager can understand. Everything else the analyst does grows out of that go-between position.

● MUST MEMORISE

ASKED ALMOST EVERY YEAR

A systems analyst is a business problem solver. Roles and responsibilities: study the existing system and business processes to find problems; gather and analyse user requirements; carry out feasibility studies; document requirements and prepare system models and specifications; guide design, testing, implementation and user training; and act as the link between users and the technical team.

Memory hook: the analyst is a **bridge**. A bridge touches both banks (users and builders) and carries traffic both ways (needs across to the builders, working systems back to the users).

"How does an analyst solve problems?" is a recurring exam question, and it wants a definite sequence, not a vague answer. The analyst first **researches and understands the problem**, then follows these steps:

1. Verify that the benefits of solving the problem outweigh the costs.
2. Define the requirements for solving the problem.
3. Develop a set of possible solutions (alternatives).
4. Decide which solution is best, and make a recommendation.
5. Define the details of the chosen solution.
6. Implement the solution.
7. Monitor to make sure the desired results are obtained.

Notice the shape: understand, justify, explore options, choose, specify, build, check. It is the same shape as the life cycle you meet in Unit 2, because the life cycle is this problem-solving sequence scaled up to a whole project.

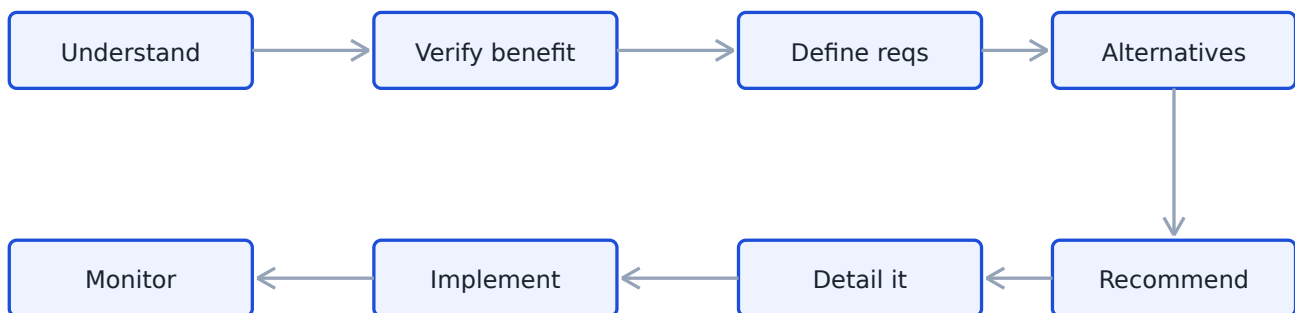


Figure 1.1 · The analyst's problem-solving sequence. It reads left to right along the top, then right to left along the bottom.

When the analyst does solve the problem with technology, the solution is usually an **information system**: a collection of interrelated components (people, data, processes and information technology) that work together to collect, process, store and output the information an organisation needs. Its job is to process input, keep data, produce reports, answer queries and handle transactions.

Any system can be broken into **subsystems**: smaller systems that are part of a larger one. A customer relationship management (CRM) system, for instance, has a sales management subsystem, an account management subsystem and a product information subsystem. Splitting a system into its components this way is called **functional decomposition**, and it is the analyst's single most useful habit of mind: it lets you focus on one area of a business, or the interface between two areas, without having to hold the whole thing in your head at once. Every diagram in this course is a form of functional decomposition.

Because the role is a bridge, the analyst needs three kinds of knowledge and skill. "Describe the skills of a systems analyst" is a standard question; answer it in these three groups and give an example in each.

- ▶ **Technical knowledge and skills.** Understanding of computers, storage, input and output, networks, programming languages and operating systems, and of the tools and techniques used to build systems, so the analyst can understand and communicate technical requirements even without doing the programming.
- ▶ **Business knowledge and skills.** Understanding of how organisations work, how they are structured and managed, and what the specific business does and what makes it succeed, so the system is aligned with real organisational goals.
- ▶ **People (interpersonal) knowledge and skills.** The most important of the three, because the analyst relies on managers, users, programmers and vendors to take a system from idea to delivery. The analyst must communicate well in interviews, reviews and presentations, manage expectations and resolve conflict.

The analyst works with both tools and techniques, and the examiner likes you to tell them apart:

- ▶ A **tool** is a software product used to develop analysis and design specifications or completed components. Examples: a modelling tool such as a computer-aided visual modelling package, a software package such as a database product, and integrated development environments (IDEs) such as Visual Studio.
- ▶ A **technique** is a strategy for completing a specific development activity. Examples: project planning techniques, cost-benefit analysis, interviewing techniques, requirements modelling, and database design techniques.

The one-line distinction: a **tool is a thing you use**; a **technique is a way of doing something**. You will meet a third and fourth term, model and methodology, in Unit 3, and the four-way distinction between model, tool, technique and methodology is itself an exam question.

1. In one sentence, what is a systems analyst?
2. Name the three kinds of skill a systems analyst needs.
3. What is functional decomposition?

ANSWERS

1. A business problem solver who studies a business, finds problems, and specifies the information system that solves them, acting as the bridge between users and the technical team.
2. Technical, business, and people (interpersonal) skills.
3. Breaking a system into smaller subsystems (its components) so each can be studied on its own.

⚠ TRAP

ANALYST IS NOT CODER

Asked "what does a systems analyst do?", weak answers say "writes the program" or "codes the system". That is the programmer. The analyst's product is **understanding and specification**: models, requirements and recommendations. Say "problem solver" and "bridge between users and technical team", and you have the marks; say "programmer" and you have missed the whole point of the course.

MODULE ONE · UNIT 2

The System Development Life Cycle

By the end of this unit you can name the phases of the life cycle, state the objective, the main activities and the deliverables of each, and list the principles of a successful systems development project.

TEACH

THE SINGLE MOST EXAMINED TOPIC

The entire process of building, deploying, using and updating an information system is called the systems development life cycle (SDLC). Businesses use information systems to support the work they have to do, and the SDLC is the orderly, phased path a project follows so that the system is built in a disciplined way, on schedule, within budget and to a standard, instead of by trial and error. Without a methodology, projects run over cost, deliver late, or produce a system that does not do what users needed.

"Discuss the major phases of the SDLC and identify the deliverables in each phase" is worth nine or ten marks and has appeared on almost every paper. It is the single most reliable question on the exam. Learn the five phases, their objectives and their deliverables cold.

The SDLC has **five phases: Planning, Analysis, Design, Implementation, and Support**. The first four are the development project; Support is what happens for years afterwards and is not usually counted as part of the original project.

1. **Planning:** identify the need for the system and decide whether it is feasible and worthwhile. Objective: identify the scope, confirm feasibility, and produce a schedule, resource plan and budget.
2. **Analysis:** study the current system and business to determine detailed requirements. Objective: understand and document what the system must do.
3. **Design:** turn requirements into detailed specifications. Objective: design the solution system (architecture, network, interfaces, database, controls).
4. **Implementation:** build, test and deploy the system. Objective: produce a reliable, fully functional system and train the users.
5. **Support:** keep the system running productively after deployment. Objective: maintain, enhance, and support the users for years.

Memory hook: P A D I S: "Please Analyse, Design, Implement, Support." Five letters, five phases, in order.

TEACH

THE ACTIVITIES INSIDE EACH PHASE

Each phase is a group of activities. You do not have to recite every one, but knowing the shape lets you write a full answer.

- ▶ **Planning:** define the problem (business problem and scope), produce the project schedule, confirm project feasibility, staff the project, and launch it.
- ▶ **Analysis:** gather information; define system requirements; prioritise requirements; build prototypes to discover requirements; generate and evaluate alternatives; review recommendations with management.
- ▶ **Design:** design the application architecture; design and integrate the network; design the user and system interfaces; design and integrate the database; design the system controls.
- ▶ **Implementation:** construct software components; verify and test; convert data; train users and document the system; install the system.
- ▶ **Support:** maintain the system (fix bugs, make minor adjustments); enhance the system (add capability as the business changes); and support the users, which includes running a **help desk**.

A **deliverable** is the concrete output a phase hands to the next. This is the part of the question most candidates forget, so it is the part that separates a six-mark answer from a ten-mark one.

PHASE	DELIVERABLES
Planning	Project proposal and recommendation; investigation report; analysis of alternatives; cost-benefit analysis; schedule.
Analysis	Detailed system requirements; revised cost-benefit analysis; revised schedule.
Design	Logical design; technical design; test plans; revised cost-benefit analysis; revised schedule.
Implementation	System elements; implementation plans; contingency plans; end-user training materials; maintenance plan; operations plans; cutover or phase-in.
Support	Maintenance reports; updated system documentation.

Installation, turning off the old system and turning on the new, sits in the Implementation phase, and there are **four conversion strategies**. They come up again in Module Six, but meet them here:

1. **Direct cutover:** the new system immediately replaces the old one. Cheapest and fastest, but riskiest: if the new system fails there is nothing to fall back on.
2. **Parallel:** both systems run together for a month or two until it is clear the new one has no bugs. Safest, but the most expensive, because everything is done twice.
3. **Phased:** small parts of the new system gradually replace small parts of the old. Lower risk, but drawn out.
4. **Pilot:** one small group of users runs the new system before everyone else. Limits the blast radius of a failure to one group.

Now you try: a hospital with a live patient-safety system is replacing it. Which conversion strategy would you recommend, and why? (Think about what "the new system fails" costs a hospital, then re-read the four options.)

"What are the principles of a successful systems development?" is a Section A question in its own right. The principles are:

- ▶ Get the user involved.
- ▶ Use a problem-solving approach.
- ▶ Establish phases and activities.
- ▶ Establish standards for development and documentation.
- ▶ Justify systems as capital investments (cost-benefit).
- ▶ Do not be afraid to cancel or revise scope.
- ▶ Divide and conquer (break the system into manageable parts).
- ▶ Design systems for growth and change.
- ▶ Apply proper planning and project management.

Memory hook: the first two are the big ones. If you remember nothing else, remember **get the user involved** and **use a problem-solving approach**: they are the two the examiner most wants to see.

RECALL

COVER THE ANSWERS

1. List the five phases of the SDLC in order.
2. Which phase produces the cost-benefit analysis and the schedule as first deliverables?
3. In which phase does installation happen, and name its four strategies.

ANSWERS

1. Planning, Analysis, Design, Implementation, Support (PADIS).
2. Planning.
3. Implementation; direct cutover, parallel, phased, pilot.

MODULE ONE · UNIT 3

Methodologies, Feasibility and Approaches

By the end of this unit you can distinguish the predictive and adaptive approaches, describe the main methodologies and say when to use each, and carry out the feasibility checks a project must pass in the Planning phase.

How a project handles requirements, change and uncertainty puts it in one of two families:

- ▶ A **predictive** (traditional) approach assumes the project can be planned in full in advance and built to that plan, progressing through a fixed sequence of phases each completed before the next begins. It suits systems that are well-understood and unlikely to change.
- ▶ An **adaptive** approach assumes the project cannot be fully planned in advance and must be modified as it goes. Development runs in short cycles (iterations or sprints), each producing a working piece of the system, with continuous user feedback. It suits changing requirements and high uncertainty.

PREDICTIVE APPROACH	ADAPTIVE APPROACH
Requirements are defined upfront	Requirements evolve over time
Development is linear and structured	Development is iterative and incremental
Emphasis on detailed documentation	Strong user involvement and collaboration
Limited flexibility to accommodate change	Working systems valued over extensive documentation

A **methodology** is a complete, named way of running a project: it bundles a life-cycle model with the tools, techniques and rules for using them. These are the methodologies the examiner names, with the one thing you must be able to say about each and when to choose it.

- ▶ **Waterfall:** the classic predictive model. Phases flow strictly downward, each finished before the next starts. Choose it when requirements are stable and well understood. Weakness: change late in the project is very costly.
- ▶ **Prototyping:** build a quick working model, show it to users, refine it. Choose it when requirements are unclear and need to be discovered. Weakness: the prototype can be mistaken for the finished system, and scope can creep.
- ▶ **Rapid application development (RAD):** fast delivery through prototyping, reusable components and heavy user involvement. Choose it when the schedule is short and requirements are unclear. Weakness: less suited to large, complex systems.
- ▶ **Spiral:** an adaptive model that repeats a loop of plan, analyse and design, build a prototype, then test and integrate, adding risk analysis each turn. Choose it for large, high-risk projects.
- ▶ **Incremental:** deliver the system in working increments, each adding capability. Choose it when you want value in users' hands early.
- ▶ **Agile:** the modern adaptive family; short iterations, continuous feedback, working software over documentation. Choose it when requirements change often.
- ▶ **V-shape:** waterfall bent into a V, pairing each development stage with the test that validates it. Choose it when verification and traceability matter.
- ▶ **Object-oriented design (OOD):** analyses and designs the system as cooperating objects rather than as separate data and processes. It underlies the UML diagrams of Module Five.

A recurring question: "You are a systems analyst on a project with unclear user requirements, a short time schedule and schedule visibility. What methodology do you recommend? Give its advantages and disadvantages."

1 Match the clues to a family.

"Unclear requirements" rules out a predictive model like waterfall, which needs requirements fixed upfront. "Short schedule" points to fast, adaptive delivery.

2 Name the methodology.

Rapid application development (RAD), or prototyping, is the textbook answer: both discover requirements as they go and deliver quickly.

3 Give advantages.

Fast delivery; requirements discovered with the user, so the result fits real needs; strong user involvement raises acceptance.

4 Give disadvantages.

Less discipline and documentation; can struggle on large complex systems; scope creep if the prototype keeps growing.

Now you try: your manager wants to replace waterfall with prototyping on a **large and complex** project. Do you agree? (Re-read RAD's weakness: what does "large and complex" do to an adaptive method?)

Now the full four-way distinction, an exam question by itself. Keep the one-line version of each:

- ▶ A **model** is a representation of an important aspect of the real world (a diagram of the system). It leaves out detail so you can reason about the whole.
- ▶ A **tool** is a software product used to help do the work (a modelling package, an IDE).
- ▶ A **technique** is a strategy for completing one activity (interviewing, cost-benefit analysis).
- ▶ A **methodology** is the complete package: a life-cycle model plus the tools, techniques and rules that say how to run the whole project (waterfall, agile).

Reading order of size: a methodology contains techniques, techniques use tools, and tools produce models.

TEACH**FEASIBILITY IN THE PLANNING PHASE**

Before a project is allowed to proceed, the Planning phase runs a **feasibility study** to decide whether the proposed system can be built and is worth building. A feasibility study reduces risk: by finding constraints early, management can proceed, modify or abandon before money is wasted. It checks a project from several angles, and each angle carries a matching risk:

- ▶ **Technical feasibility:** is the technology and skill available? Risk: the team cannot build what is proposed.
- ▶ **Economic feasibility:** do benefits outweigh costs (cost-benefit analysis)? Risk: the system costs more than it saves.
- ▶ **Operational feasibility:** will users accept and use it, and does it fit the organisation? Risk: a working system nobody adopts.
- ▶ **Schedule feasibility:** can it be finished in the time required? Risk: delivered too late to be useful.
- ▶ **Legal feasibility:** does it comply with laws and regulations? Risk: fines or forced shutdown.

● MUST MEMORISE**FIVE FEASIBILITIES**

The five feasibility types: **Technical, Economic, Operational, Schedule, Legal.**

Memory hook: TEOSL: or read it as "The Exam Only Suits the Legal", any sentence that fixes the five initials. Note that "confirming project feasibility" is one of the five Planning activities from Unit 2, so feasibility links the two units.

RECALL**COVER THE ANSWERS**

1. Give the one-line difference between the predictive and adaptive approaches.
2. Which methodology suits unclear requirements and a short schedule?
3. Name the five feasibility types.

ANSWERS

1. Predictive plans the whole project upfront and builds to the plan; adaptive plans as it goes, in short iterations with user feedback.
2. RAD (rapid application development), or prototyping.
3. Technical, economic, operational, schedule, legal.

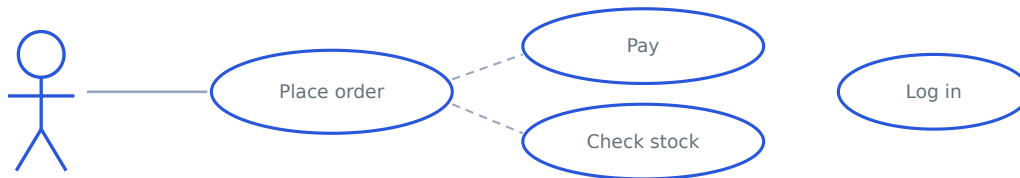
⚠ TRAP**NAME THE METHODOLOGY, THEN JUDGE**

When a question describes a situation and asks which methodology, weak answers list every methodology they know. The marks are for **choosing one** that fits the clues in the scenario and **justifying it from those clues**, then giving its advantages and disadvantages. Read the scenario for the tell-tale words, "unclear requirements", "short schedule", "large and complex", "high risk", "stable requirements", and let them pick the methodology for you.

MODULE TWO

Requirements: Discovery and Modelling

A system can only be as good as the understanding it is built on. This module is the Analysis phase in detail: how to find out what a business really needs, how to write those needs down so nothing is lost, and how to draw the first model of the system, the use case diagram.



Fact-Finding and Requirements

By the end of this unit you can tell a functional requirement from a non-functional one, name and justify the requirement elicitation techniques with their stakeholders and limitations, and explain the as-is and to-be view of a system.

TEACH

WHAT A REQUIREMENT IS

A **requirement** is a statement of something the system must do or a quality it must have. The whole of the Analysis phase exists to discover, agree and record requirements, because everything designed and built afterwards is built to satisfy them. Get the requirements wrong and every later phase faithfully builds the wrong system. Requirements are divided two ways, and the examiner asks about both.

By **what they describe**:

- ▶ A **functional requirement** is something the system must *do*: a function or feature. "The system shall let a customer place an order." "The system shall calculate the total of an order." They map directly onto use cases and processes.
- ▶ A **non-functional requirement** is a *quality* the system must have: how well it does things, not what it does. Performance, security, usability, reliability, availability. "The system shall respond within two seconds." "The system shall encrypt all passwords."

● MUST MEMORISE

FUNCTIONAL VS NON-FUNCTIONAL

Functional = what it does (a verb). Non-functional = how well it does it (a quality). The paper asks you to formulate three functional and two non-functional requirements from a scenario. Functional ones start "the system shall [verb]"; non-functional ones name a quality (fast, secure, reliable, usable, available).

Memory hook: non-functional requirements are the "-ilities": security, reliability, usability, scalability, availability.

TEACH

THREE LEVELS OF REQUIREMENT

By **whose language they are in**, requirements come at three levels. The 25/26 paper asks you to formulate all three from one scenario, so keep them separate:

- ▶ **Business requirements:** the high-level goals of the organisation. "Reduce the time to process an admission by half." Written in the language of the business.
- ▶ **User requirements:** what a particular user needs to do with the system. "A nurse needs to record a patient's vitals at the bedside." Written from the user's point of view, often as user stories.
- ▶ **System requirements:** the detailed, technical statements the developers build to. "The system shall store vitals as time-stamped records linked to a patient ID." These are the functional and non-functional requirements above.

They nest: business goals drive user needs, which drive system requirements.

Analysis studies two versions of the system. The **as-is** system is the current one, with all its weaknesses; the **to-be** system is the proposed one, with the improvements. A common exam task is to "identify the weaknesses of the as-is system, the expected improvements in the to-be system, and the assumptions that must be validated." Answer it in exactly those three parts: list what is wrong now, list what the new system will fix, and list the things you are assuming that had better be true (for example, that users have smartphones, or that the network reaches every ward).

Discovering requirements is called **fact-finding**, **requirements elicitation**, or **gathering information**. The examiner asks you to choose two or three techniques and, for each, justify **why** it suits this project, say **what** information it collects and from **whom**, and give a limitation. Learn the techniques with those hooks:

TECHNIQUE	WHAT / WHO	A LIMITATION
Interview	One-to-one questions with a key user or manager; rich, deep information, can follow up.	Time-consuming; quality depends on the interviewer; only a few people.
Questionnaire	Written questions to many people at once; cheap way to reach a large, scattered group.	No follow-up; low response rate; answers can be shallow.
Observation	Watching users actually do the work; reveals what people really do, not what they say.	People behave differently when watched; time-consuming; a snapshot.
Document review	Reading existing forms, reports, procedures and business rules; grounds you in facts.	Documents may be out of date or not match real practice.
JAD	Joint application development (JAD): a facilitated workshop bringing users and analysts together; fast consensus.	Hard to schedule; needs a skilled facilitator; dominant voices can skew it.
Prototyping	Building a quick model for users to react to; surfaces requirements people cannot state in the abstract.	Users may think it is finished; can encourage scope creep.

The 25/26 paper asks for a full **requirements elicitation plan**: for each technique, the stakeholders involved, the information collected, and why it suits the project. Suppose the project is a new library management system.

- 1 Pick techniques that cover different sources.
One for depth (interview), one for breadth (questionnaire), one for reality (observation), one for facts (document review).
- 2 Interview the head librarian.
Information: the rules for borrowing, reservations and fines. Why: they hold the business rules and can be probed in depth.
- 3 Questionnaire to student borrowers.
Information: what they want from self-service. Why: cheaply reaches a large, scattered user group.
- 4 Observe the issue desk at a busy hour.
Information: the real check-out workflow and its bottlenecks. Why: shows what staff actually do, not what they report.
- 5 Review existing forms and the current catalogue.
Information: the data already held on books and members. Why: grounds the new system in real records.

Now you try: write the same four-line plan for a hospital appointment system. Who do you interview, who gets the questionnaire, what do you observe, and which documents do you review?

Two smaller questions round the unit off. The **goals of requirement analysis** are to understand the business need, to define what the system must do, and to produce a clear, agreed, prioritised set of requirements. The **overall strategy for fact-finding** is the sensible order of work: start with document review and background reading to get oriented, then observe, then interview key people, then broaden with questionnaires, and confirm findings with the users before you model. Start broad and cheap, then go deep and targeted.

1. Classify: "the system shall email a receipt" and "the system shall be available 99.9% of the time".
2. Which technique reaches many scattered people cheaply, and what is its main weakness?
3. What are the as-is and to-be systems?

ANSWERS

1. The first is functional (a thing it does); the second is non-functional (a quality, availability).
2. A questionnaire; its weakness is no follow-up and often a low response rate.
3. The as-is is the current system with its weaknesses; the to-be is the proposed improved system.

MODULE TWO • UNIT 2

Use Case Modelling

By the end of this unit you can draw a use case diagram from a scenario, use the four relationships correctly, and write a use case narrative. This is the first Section B diagram, and it appears almost every year.

TEACH

ACTORS AND USE CASES

A **use case** is a single thing a user wants to achieve with the system, named as a verb phrase: "Place order", "Cancel appointment", "Print report". A **use case diagram** shows all the use cases of a system and who uses them. It has three pieces:

- ▶ An **actor**, drawn as a stick figure: a role outside the system that interacts with it (a customer, a store keeper, an insurance carrier). An actor is a role, not a person, and can be another system.
- ▶ A **use case**, drawn as an oval with the verb phrase inside.
- ▶ The **system boundary**, a box drawn around all the use cases, with the actors outside it.

A plain line (an **association**) joins an actor to each use case it takes part in. That is the fourth relationship, and the simplest.

"Identify the types of relationships in use cases" is worth marks every year. There are four, and the two dashed ones are where marks are won and lost:

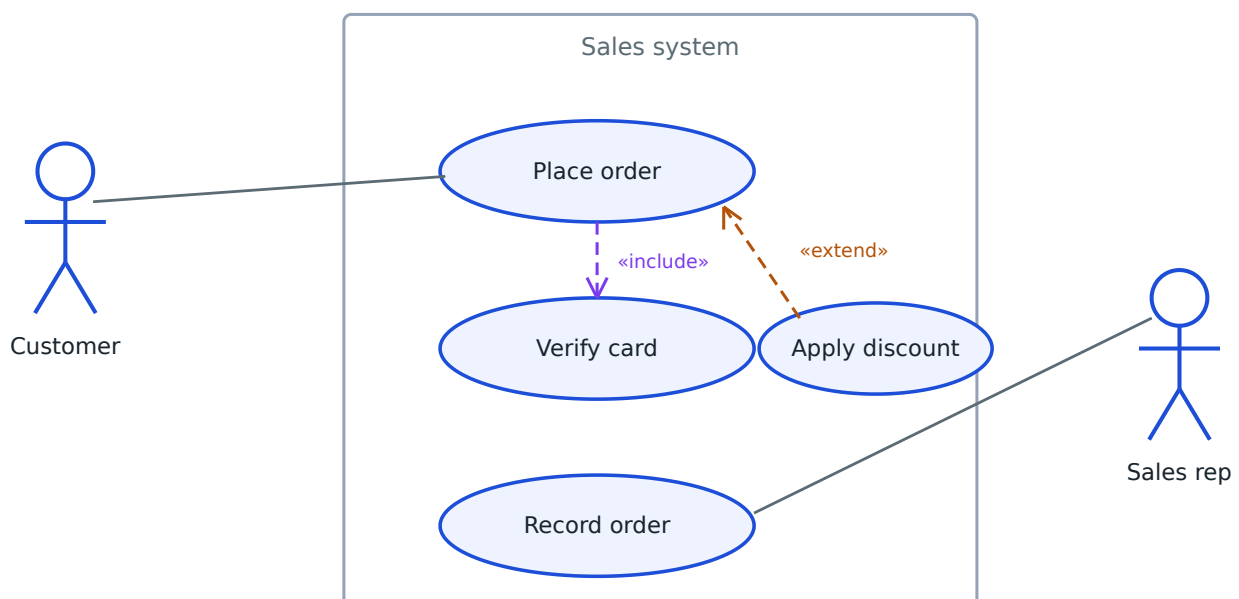
- ▶ **Association** (solid line): joins an actor to a use case it participates in.
- ▶ **Include** (dashed arrow labelled «include»): one use case *always* uses another as a sub-step. "Place order" **includes** "Verify card": the check happens every time. The arrow points to the included use case.
- ▶ **Extend** (dashed arrow labelled «extend»): one use case *optionally* adds behaviour to another under some condition. "Place order" is **extended by** "Apply discount": only sometimes. The arrow points from the extension to the base use case.
- ▶ **Generalization** (solid line with a hollow triangle): one actor or use case is a special kind of another. A "New patient" and an "Old patient" are both a "Patient"; the triangle points to the general one.

Memory hook: Include is always, Extend is maybe. The include arrow points to the step that always runs; the extend arrow points back to the base it might add to.

⚠ TRAP

WHICH WAY DOES THE ARROW POINT?

The commonest use case mistake is the direction of the include and extend arrows. Fix them with one sentence each: **the include arrow points at the thing always done** (base → included), and **the extend arrow points at the thing being added to** (extension → base). Draw both dashed, both labelled with the stereotype in guillemets, or the examiner cannot tell them from an association.



include (Verify card always runs); the amber dashed arrow is an *extend* (Apply discount runs only sometimes).

WORKED

BUILDING THE DIAGRAM FROM WORDS

A recurring scenario: *a sales management system lets a sales representative record orders placed by customers and check out orders, and lets a store keeper keep inventory. Recording an order includes updating the order; checkout extends edit-order; keeping inventory includes updating product details.* Draw the use case diagram.

1 Find the actors.

The roles outside the system that use it: Sales representative and Store keeper. Draw them as stick figures outside the boundary.

2 Find the use cases (the verbs).

Record order, Check out order, Update order, Edit order, Keep inventory, Update product details. Draw each as an oval inside the boundary.

3 Join actors to their use cases with associations.

Sales rep to Record order and Check out order; Store keeper to Keep inventory.

4 Add the include relationships (always).

Record order «include» Update order; Keep inventory «include» Update product details. Dashed arrows to the included case.

5 Add the extend relationship (maybe).

Check out order «extend» Edit order: the dashed arrow points from Check out to the base it extends.

Now you try: redraw it for the video store, where a member can Rent video (which includes Check for overdue) and Return video, and a manager can Print overdue report. Which relationship is the include?

TEACH

THE USE CASE NARRATIVE

A diagram shows *which* use cases exist; a **use case narrative** (or use case description) tells the story of *one* of them step by step. A simple narrative has: the use case name, the actor, a precondition, the main flow (the normal sequence of steps), and alternative flows (what happens when something is different, for example the card fails). You write a narrative when a question says "describe" or "give the flow of" a use case rather than "draw".

RECALL

COVER THE ANSWERS

1. Name the four use case relationships.

2. "Withdraw cash" always runs "Authenticate". Which relationship, and which way does the arrow point?

3. What does a use case narrative add that the diagram does not?

ANSWERS

1. Association, include, extend, generalization.

2. Include; the dashed arrow points from "Withdraw cash" to "Authenticate" (the step always done).

3. The step-by-step flow of one use case (precondition, main flow, alternatives).

MODULE THREE

Process and Data Flow Modelling

The data flow diagram is the workhorse of Section B. It shows how data moves through a system: who sends it in, which processes transform it, where it is stored, and what comes out. Learn its four symbols, its three levels, and the handful of mistakes the examiner hunts for, and this becomes reliable, drawable marks.



Data Flow Diagrams

By the end of this unit you can draw the four elements of a data flow diagram, build a context diagram, a Level 0 and a Level 1 from a scenario, and name and avoid the mechanical errors and illegal flows the examiner tests.

TEACH

WHAT PROCESS MODELLING IS

Process modelling shows how a system works by following the data. The tool for it is the **data flow diagram (DFD)**: a picture of how data moves from the outside world, through the processes that transform it, into and out of the places it is stored, and back out to the world. Where a use case diagram shows *what* the system does for its users, a DFD shows *how* data flows to make that happen.

The examiner also asks why a DFD is worth drawing before you build. Good reasons: it is a clear picture that non-technical users can check; it separates *what* the system does from *how* it is built; it shows the boundary of the system and every thing it touches; and it breaks a large system into manageable processes (functional decomposition again).

MUST MEMORISE

THE FOUR ELEMENTS

A DFD has exactly **four** elements. Draw each one exactly right; a wrong symbol loses the mark however good the logic.

- ▶ **External entity** (a **rectangle**): a person, department or other system *outside* the system that sends data in or receives data out. Named with a noun (Customer, Bank).
- ▶ **Process** (a **circle** or rounded rectangle, numbered): work that transforms data. Named with a verb phrase and given a number (1.0 Verify order).
- ▶ **Data store** (an **open-ended rectangle**, two parallel lines, labelled D1, D2): data at rest, a file or table. Named with a noun (D1 Customers).
- ▶ **Data flow** (an **arrow**): data in motion between the other elements. Named with the data it carries (order details), not with a verb.



Figure 3.1 · The four DFD symbols. Learn to draw all four; the data store is the one candidates get wrong (it is open at the right, not a closed box).

A DFD is drawn at levels, each an explosion of the one above. The terminology is examinable and slightly confusing, so pin it down:

- ▶ The **context diagram** shows the *whole system as a single process*, numbered 0, with the external entities around it and the net data flows in and out. It has **no data stores**. It defines the boundary: everything inside the one circle is the system, everything drawn as a rectangle is outside it.
- ▶ The **Level 0 DFD** (also called **Diagram 0**, and in some papers the **Process 0 diagram**) explodes that single process into the *major processes* of the system, numbered 1.0, 2.0, 3.0, and adds the data stores and the flows between them.
- ▶ A **Level 1 DFD** explodes *one* of those processes into its sub-processes. A Level 1 of process 3.0 has processes numbered 3.1, 3.2, 3.3.

So when a paper says "draw a context diagram, a Level 0 and a Level 1 for the rent payment process", it wants three diagrams: the system as one bubble, then its major processes, then a close-up of the rent-payment process alone.

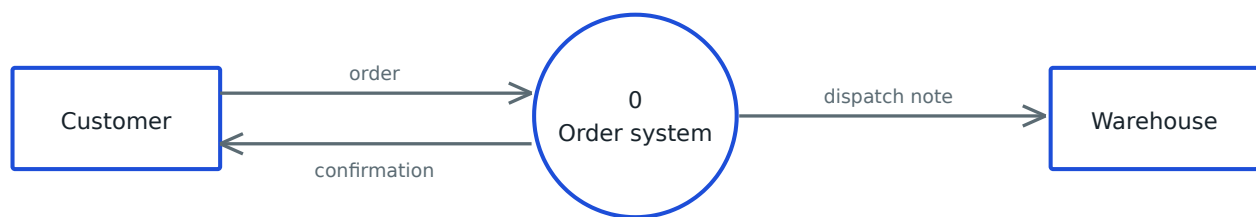


Figure 3.2 · A context diagram: the entire system is one process (numbered 0), external entities are rectangles, and only the net flows cross the boundary. No data stores appear at this level.

● MUST MEMORISE

ILLEGAL FLOWS

A data flow shows data moving from one element to another, but **a process must be at one end of every flow**. Data cannot move on its own; something has to do the moving. So these four flows are **illegal**, and drawing one is a classic lost mark:

- ▶ entity → entity (two outsiders talking bypass the system entirely)
- ▶ entity → data store (nothing processed the data before storing it)
- ▶ data store → data store (data copying itself with no process)
- ▶ data store → entity (data leaving store straight to the outside with no process)

The rule in one line: **every data flow must touch a process at one end**. An entity talks to a process, a process talks to a store, a process talks to an entity; never store-to-store, store-to-entity, or entity-to-entity, or entity-to-store.

⚠ TRAP

DATA DOES NOT MOVE ITSELF

The sentence the paper gives you, "*a data flow shows data movement from one system entity to another*," is true but incomplete, and the follow-up "*data cannot move between entities in a data flow diagram*" catches people out. What it means: data never flows straight between two **external entities**, or straight into or out of a **store** without a process. If your diagram has an arrow from a rectangle to a rectangle, or from a data store straight to a rectangle, it is wrong. Put a process bubble in the middle.

● MUST MEMORISE

MECHANICAL ERRORS

"What mechanical errors can result while constructing a DFD?" is a standard 3-mark question. There are three, all about a process whose inputs and outputs do not match:

- ▶ **Black hole:** a process with input but **no output**. Data goes in and vanishes. (A process that stores data but produces nothing.)
- ▶ **Miracle** (spontaneous generation): a process with output but **no input**. Data appears from nowhere.
- ▶ **Grey hole:** a process whose **inputs are insufficient** to produce its output. Something needed is missing (a receipt that shows a price, from an input that never carried a price).

Memory hook: in goes nothing, out comes something = miracle; in goes something, out comes nothing = black hole; in goes too little = grey hole.

TEACH

DATA BALANCING

When you explode a process into a lower level, the data flowing into and out of the exploded process on the parent diagram must be exactly the data flowing into and out of its child diagram. This rule is **data balancing** (or balancing the DFD). If process 3.0 receives "payment" and sends out "receipt" on the Level 0, then its Level 1 must also receive "payment" and, in total, send out "receipt", no more and no less. An unbalanced DFD, where a flow appears or disappears between levels, is a mechanical error, and checking the balance is one way to **validate the diagram's integrity** after you draw it, along with checking that every process has both inputs and outputs and that no flow is illegal.

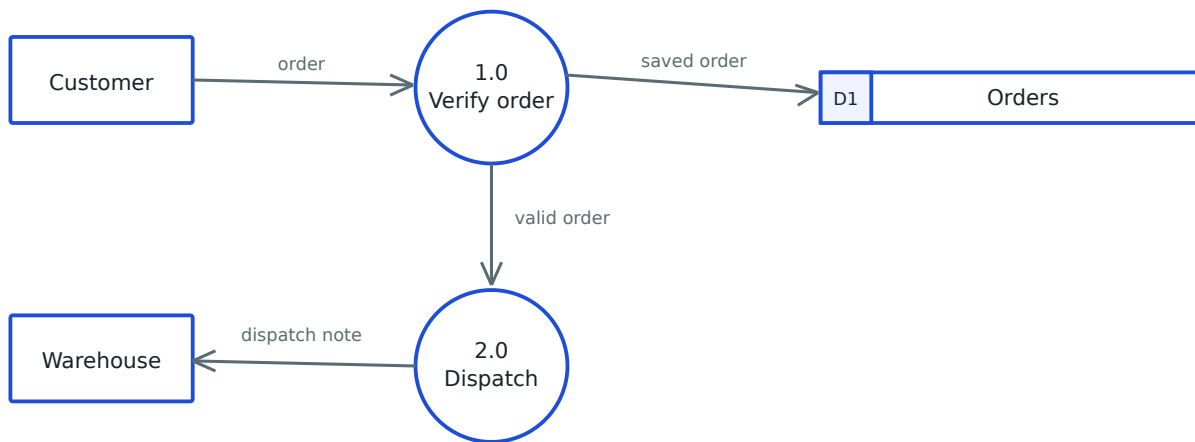


Figure 3.3 · A Level 0 DFD: the single process 0 has exploded into processes 1.0 and 2.0, a data store D1 has appeared, and every flow touches a process at one end. Balance check: the "order" in and "dispatch note" out match the context diagram.

WORKED

FROM A SCENARIO TO THREE DIAGRAMS

The routine for "draw a context diagram, a Level 0 and a Level 1" is always the same. Take the warehouse scenario: trucks arrive, drivers submit trucks, the manager tells a labourer to unload, the storekeeper takes inventory and generates a stock report for the manager.

- 1 Find the external entities (who is outside sending or receiving).
Truck driver, Warehouse manager. They become rectangles.
- 2 Context diagram: one process for the whole system.
One bubble "0 Warehouse system"; arrows in (truck, activity report) and out (stock report). No data stores yet.
- 3 Level 0: explode into the major processes.
1.0 Unload truck, 2.0 Take inventory, 3.0 Generate report; add data store D1 Stock; wire the flows so each process has input and output.
- 4 Level 1: explode one process, say 2.0 Take inventory.
2.1 Count stock, 2.2 Classify condition, 2.3 Update stock record; number them 2.x and balance the flows to match process 2.0.
- 5 Validate.
Every process has an input and an output (no black holes or miracles); no illegal flows; the levels balance.

Now you try: draw the context diagram for food ordering at a Nigerian airport, then give four reasons a DFD is helpful for modelling those processes before you build.

- 1. Name the four elements of a DFD and the shape of each.**
 - 2. A process has an input but produces no output. Which mechanical error?**
 - 3. Why can you not draw an arrow straight from a data store to an external entity?**
-

ANSWERS

1. External entity (rectangle), process (numbered circle), data store (open-ended rectangle), data flow (arrow).
2. A black hole.
3. Every data flow must touch a process at one end; a store cannot send data to the outside without a process to move it.

MODULE FOUR

Data Modelling with Entity-Relationship Diagrams

Where a data flow diagram shows data moving, an entity-relationship diagram shows data at rest: the things a business keeps facts about and how they connect. This is the most heavily drawn topic on the paper. The text teaches it in Chen notation; the exam asks for the crow's foot, so learn both symbols cold.



The Entity-Relationship Model

By the end of this unit you can draw an entity-relationship diagram from a narrative, read cardinality off a diagram, use the crow's-foot and Chen notations, choose an identifier, and interpret a relationship by forward and backward chaining.

TEACH

TWO NOTATIONS, ONE FOR THE EXAM

The entity-relationship diagram is examined heavily and is taught in the course text, but in the **Chen** style (relationships as diamonds, attributes as ovals). The exam, by contrast, asks you to draw it in the **Information Engineering** style (the crow's foot). This unit teaches both and says clearly which the paper wants, so you are never caught out. Read it slowly and draw every figure yourself.

The **entity-relationship (E-R) model** describes the data a business needs to store as a set of **entities** and the **relationships** between them. Drawn out, it is an **entity-relationship diagram (ERD)**. It answers the question "what things does this business keep information about, and how are they connected?" It says nothing about process; that is the data flow diagram's job. This is the single difference the examiner asks for between a data model and a process model.

MUST MEMORISE

THE ELEMENTS OF AN ERD

An ERD is built from four things:

- ▶ **Entity:** a thing the business stores data about (Customer, Order, Patient). Drawn as a **rectangle** with the entity name, singular, in capitals.
- ▶ **Attribute:** a fact about an entity (a customer's name, an order's date). In crow's-foot notation the attributes are listed inside the entity's rectangle.
- ▶ **Identifier (or key):** the attribute that uniquely identifies each instance of an entity. The chosen one is the **primary key (PK)**. When an entity holds the key of another to record a relationship, that borrowed key is a **foreign key (FK)**.
- ▶ **Relationship:** an association between entities, drawn as a **line** with a verb naming it (a customer *places* an order).

TEACH

CHOOSING AN IDENTIFIER

The paper asks you to give "two options for an identifier for a student entity and justify your choice." An identifier must be **unique** and **unchanging**. For a student, the two options are typically the **matriculation number** and the **email address**. Justify: the matriculation number is best because it is assigned once, never reused, and never changes; an email can be changed or reassigned, so it is a weaker key. The lesson generalises: prefer a stable, system-assigned identifier over one the user can change.

● MUST MEMORISE

CARDINALITY: THE CARDINAL RELATIONSHIPS

Cardinality is how many instances of one entity relate to how many of another. There are three cardinal relationships, and naming and drawing them is worth marks every year:

- ▶ **One-to-one (1:1):** each instance of A relates to at most one of B, and vice versa. One employee manages one department, and one department has one manager.
- ▶ **One-to-many (1:M):** one instance of A relates to many of B, but each B to one A. One customer places many orders, but each order belongs to one customer.
- ▶ **Many-to-many (M:N):** each A relates to many B and each B to many A. A student takes many courses, and each course has many students.

Memory hook: read the sentence both ways. "One customer places many orders" and "one order is placed by one customer" gives you 1:M. Always test a relationship in both directions; that is what tells you the cardinality.

TEACH

CARDINALITY VERSUS ORDINALITY

The paper asks you to "differentiate between cardinality and ordinality." Keep them apart:

- ▶ **Cardinality** is the **maximum** number on each side: one or many. It is the 1, the M and the N above.
- ▶ **Ordinality** (also called optionality or the minimum) is whether the relationship is **mandatory or optional**: must there be at least one, or is zero allowed? "An order must have a customer" is mandatory (minimum one); "a customer may have no orders yet" is optional (minimum zero).

Together they give the full rule: a customer relates to **zero or many** orders (optional, many), an order relates to **exactly one** customer (mandatory, one).

● MUST MEMORISE

CROW'S-FOOT SYMBOLS

The **Information Engineering** style, the **crow's-foot** notation, puts the cardinality and ordinality at the ends of the relationship line with two marks: an inner mark for the minimum and an outer mark for the maximum. Learn the four combinations.

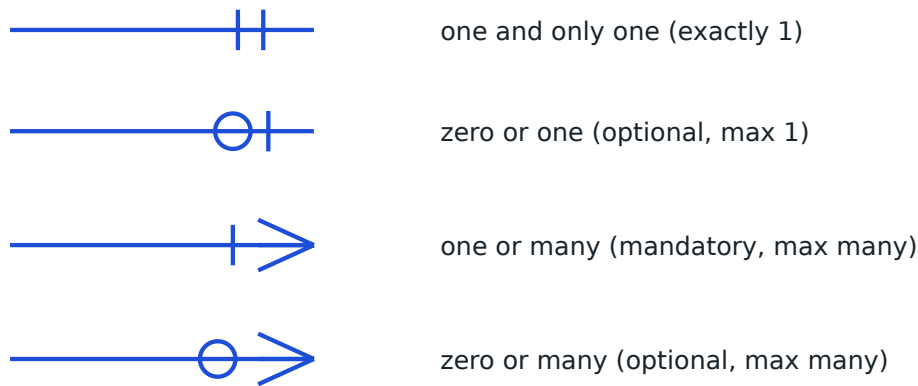


Figure 4.1 · Crow's-foot cardinality. The bar means "one", the circle means "zero" (optional), and the crow's foot means "many". The inner mark is the minimum, the outer mark the maximum.

TEACH

THE NOTATION STYLES

Three notations appear in the papers. They say the same things with different symbols, and you should be able to use whichever a question asks for:

- ▶ **Information Engineering (crow's foot):** entities are rectangles with attributes listed inside; cardinality is the bar-circle-crowsfoot marks above. This is the style the current paper asks for ("using Information Engineering style") and the one to default to.
- ▶ **Martin's notation:** a close cousin of crow's foot, also using the crow's-foot marks for "many"; the paper sometimes says "the Martin's ERD notation style."
- ▶ **Chen notation:** the original style. Entities are rectangles, but each **relationship is a diamond** on the line, and **attributes are ovals** hung off the entity. Cardinality is written as 1, M or N beside the line rather than as symbols.

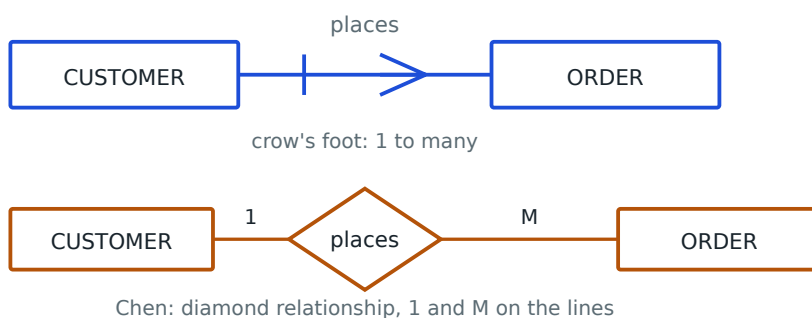


Figure 4.2 · The same 1:M relationship in crow's-foot (top) and Chen (bottom) notation. Chen puts the relationship in a diamond and writes the numbers on the lines; crow's foot puts marks at the ends.

Take the patient billing scenario: *a patient must be assigned to only one doctor, but a doctor can have many patients; a patient can have only one insurance ID.* Draw the ERD.

- 1 Find the entities (the nouns kept as records).
PATIENT, DOCTOR, INSURANCE. Three rectangles.
- 2 Find the relationships (the verbs).
A doctor treats patients; a patient has insurance. Draw a line for each with the verb on it.
- 3 Work out the cardinality of each, both ways.
"One doctor, many patients" and "each patient, one doctor" gives DOCTOR 1:M PATIENT. "A patient has only one insurance" gives PATIENT 1:1 INSURANCE (or M:1 if insurers are shared, but the fraud rule fixes it at one each).
- 4 Put the crow's-foot marks on.
On the DOCTOR-PATIENT line: one bar at DOCTOR (one), a crow's foot at PATIENT (many). On PATIENT-INSURANCE: a bar at each end (one to one).
- 5 Choose the keys.
patientId (PK) in PATIENT, doctorId (PK) in DOCTOR; PATIENT holds doctorId and insuranceId as foreign keys (FK) to record its relationships.

Now you try: KHMS schools have one or more annexes; each annex has one or more teachers, but a teacher is assigned to one annex; each teacher coordinates one or more classes, but a class is coordinated by one teacher. Draw the ERD, marking every cardinality.

TEACH

READING AN ERD: FORWARD AND BACKWARD CHAINING

The 25/26 paper prints an ERD and asks you to "read and interpret it using forward and backward chaining", and to read the cardinal relationships between the entities. Reading an ERD means turning its symbols back into English sentences, one relationship at a time, in both directions:

- ▶ **Forward chaining** reads the relationship in the direction of the verb, from the first entity to the second: "a CUSTOMER *places* zero or many ORDERs."
- ▶ **Backward chaining** reads it the other way: "an ORDER *is placed by* one and only one CUSTOMER."

Do this for every line and you have interpreted the whole diagram. The cardinality you report for each relationship is exactly what the crow's-foot marks say: look at the mark nearest each entity, translate it (bar = one, foot = many, circle = optional), and write the sentence.

⚠ TRAP

M:N NEEDS A LINK, AND THE VERB BELONGS ON THE LINE

Two ERD mistakes cost marks. First, a **many-to-many** relationship that carries its own data (a STUDENT takes a COURSE *with a grade*) cannot store that data on either entity: the grade belongs to the pairing. In logical design this becomes a **linking (associative) entity** between them, turning one M:N into two 1:M. Second, the **relationship name is a verb and sits on the line**, not inside a box; leaving a line unlabelled, or writing a noun on it, loses the reading marks.

- 1. Give the three cardinal relationships and an example of each.**
 - 2. What is the difference between cardinality and ordinality?**
 - 3. In crow's foot, what do a bar, a circle and a crow's foot each mean?**
-

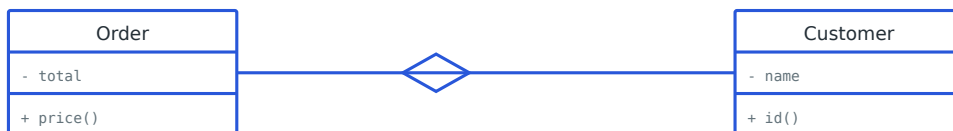
ANSWERS

1. One-to-one (employee manages department), one-to-many (customer places orders), many-to-many (student takes courses).
2. Cardinality is the maximum (one or many); ordinality is the minimum, whether the relationship is mandatory (at least one) or optional (zero allowed).
3. Bar = one, circle = zero (optional), crow's foot = many.

MODULE FIVE

Object-Oriented Modelling with UML

The last family of diagrams, and the richest source of Section B marks outside the ERD. This module teaches the Unified Modeling Language: the class diagram and its relationships, the object-oriented concepts the examiner names every year, and the two behavioural diagrams, activity and sequence. All of it is in the course text; this module makes it drawable under exam time.



UML and Class Diagrams

By the end of this unit you can say what UML is and who created it, sort UML diagrams into the two categories, draw a class with its attributes and methods, show relationships and multiplicities, and tell an object from a class.

TEACH

WHAT UML IS

The **Unified Modeling Language (UML)** is a standard set of diagram notations for modelling object-oriented systems. It was created in the 1990s by **Grady Booch, James Rumbaugh and Ivar Jacobson**, often called the "three amigos", who merged their separate methods into one, and it is maintained by the Object Management Group. Its uses: to visualise, specify, construct and document the parts of a software system, so that analysts, designers and programmers share one language.

The paper asks you to "write short notes on UML, stating its inventors and its uses." That is the paragraph above: what it is, the three inventors, and the four uses (visualise, specify, construct, document).

MUST MEMORISE

THE TWO UML CATEGORIES

The paper asks you to "categorise UML diagrams into two broad categories, mention two diagrams in each, and describe their function." The two categories are:

- ▶ **Structural diagrams** show the static structure, the things that make up the system and how they are arranged. Two examples: the **class diagram** (classes and their relationships) and the **object diagram** (objects at a moment in time). Their function is to model what the system *is*.
- ▶ **Behavioural diagrams** show the dynamic behaviour, how the system acts over time. Two examples: the **use case diagram** (what users can do) and the **activity diagram** (the flow of a process); the **sequence diagram** is a third. Their function is to model what the system *does*.

Memory hook: structure is a noun, behaviour is a verb. Class and object diagrams are nouns (things); use case, activity and sequence diagrams are verbs (happenings).

TEACH

OBJECT VERSUS CLASS

"Differentiate between an object and a class" is a standard 2-mark question. A **class** is a blueprint or template that defines the attributes and methods a kind of thing has; an **object** is a specific instance created from that class, with real values. *Car* is a class; *your particular car*, with its registration and colour, is an object. One class, many objects. This idea comes from object-oriented programming (OOP), where a program is built from objects that hold data and behaviour together.

A **class** is drawn as a rectangle in **three compartments**: the class **name** at the top, its **attributes** (data) in the middle, and its **methods** or operations (behaviour) at the bottom. A **visibility** mark sits before each member: **minus** for private (hidden inside the class), **plus** for public (visible outside). The paper asks you to "draw a class with at least two attributes and two methods", so practise the box until it is automatic.

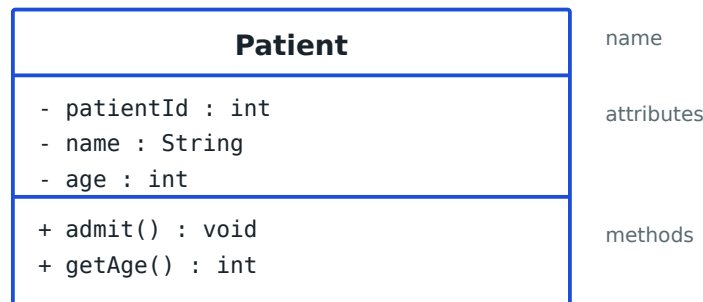


Figure 5.1 · A class box. Name, then attributes, then methods. Minus is private, plus is public. This is the unit of a class diagram.

Classes are joined by relationships, each with a different line-end, and each line carries a **multiplicity** saying how many objects take part. Multiplicity is the class-diagram version of cardinality: **1** (exactly one), **0..1** (optional), ***** or **0..*** (many), **1..*** (one or more). The relationships:

- ▶ **Association** (plain line): the classes are connected. A ward *has* patients.
- ▶ **Aggregation** (hollow diamond at the whole): a "has-a" where the part can exist without the whole. A department aggregates lecturers; the lecturers survive if the department closes.
- ▶ **Composition** (filled diamond at the whole): a stronger "has-a" where the part cannot exist without the whole. A ward is composed of a convenience room; no ward, no room.
- ▶ **Generalization** / inheritance (hollow triangle at the parent): an "is-a". A new patient and an old patient are both a patient.

Memory hook: hollow diamond is a loose has-a, filled diamond is a life-or-death has-a, hollow triangle is an is-a.

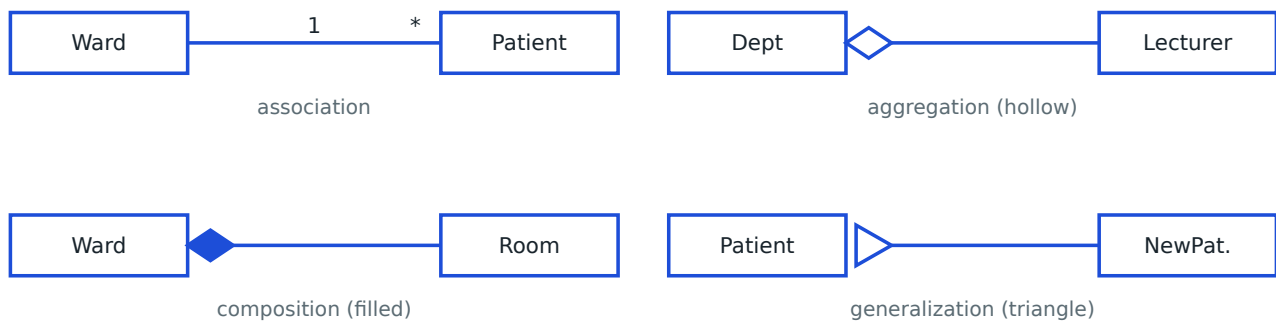


Figure 5.2 · The four class relationships. Hollow diamond = aggregation, filled diamond = composition, hollow triangle = generalization, plain line = association. The 1 and * are multiplicities.

WORKED

A CLASS DIAGRAM FROM A SCENARIO

Take the hospital scenario: *a patient is made up of one patient detail (Pid, name, age, gender); a patient is either a new patient or an old patient; a ward holds at least 8 patients; a ward is made up of one ward detail (WardNo, No_of_Bed, WType); a convenience room is part of a ward; a ward has at least two nurses.* Draw the class diagram with relationships and multiplicities.

- 1 Make a class for each noun.
Patient, PatientDetail, NewPatient, OldPatient, Ward, WardDetail, ConvenienceRoom, Nurse. Give each its attributes.
- 2 "Is made up of one detail" is composition.
Patient has a filled diamond to PatientDetail (1); Ward has a filled diamond to WardDetail (1). The detail cannot exist without its owner.
- 3 "Either a new or an old patient" is generalization.
NewPatient and OldPatient both point with a hollow triangle to Patient.
- 4 "A ward holds at least 8 patients", "at least two nurses" are associations with multiplicity.
Ward to Patient marked 1 on the ward end and 8.. on the patient end; Ward to Nurse marked 2..*.*
- 5 "A convenience room is part of a ward" is composition or aggregation.
"Part of" a ward, and it makes no sense without the ward, so a filled diamond from Ward to ConvenienceRoom.

Now you try: draw the class diagram for an order system where an order is made up of three order items, an order is either a special order or a normal order, and a customer places at most 20 orders. Which relationship is each?

1. Who invented UML, and name the two diagram categories.
2. What are the three compartments of a class box?
3. Which line-end means composition, and what does composition imply?

ANSWERS

1. Booch, Rumbaugh and Jacobson; structural and behavioural diagrams.
2. Name, attributes, methods.
3. A filled diamond at the whole; the part cannot exist without the whole.

MODULE FIVE · UNIT 2

Object-Oriented Design Concepts

By the end of this unit you can explain encapsulation, inheritance, polymorphism, composition, aggregation and association with an example each, and state the drawbacks of multiple inheritance.

TEACH

THE CONCEPTS, WITH AN EXAMPLE EACH

"Briefly describe the following object-oriented design concepts with relevant examples" is on almost every paper. Learn one clean sentence and one example for each:

- ▶ **Encapsulation:** bundling data and the methods that act on it inside one class, and hiding the data behind those methods (private attributes, public methods). Example: a BankAccount hides its balance and exposes deposit() and withdraw().
- ▶ **Inheritance:** a class (the child or subclass) takes on the attributes and methods of another (the parent or superclass) and can add its own. Example: a SavingsAccount inherits from Account and adds interest.
- ▶ **Polymorphism:** the same method name behaves differently on different classes. Example: draw() on a Circle and draw() on a Square both draw, each in its own way.
- ▶ **Composition:** an object is built from other objects that cannot exist without it (a strong "has-a"). Example: a House is composed of Rooms.
- ▶ **Aggregation:** an object is built from other objects that can exist independently (a weak "has-a"). Example: a Team aggregates Players, who exist without the team.
- ▶ **Association:** two classes are related and can use each other, but neither owns the other. Example: a Doctor treats a Patient.

● MUST MEMORISE

COMPOSITION VS AGGREGATION

The examiner loves to test composition against aggregation because they look almost the same. The one distinction: **composition is a life-or-death has-a** (the part dies with the whole, filled diamond); **aggregation is a loose has-a** (the part lives on, hollow diamond). House and Rooms is composition; Team and Players is aggregation.

Memory hook: break a House and the Rooms go with it (composition); disband a Team and the Players walk away (aggregation).

Multiple inheritance is when one class inherits from **more than one** parent at once. It is powerful but the paper asks for its **drawbacks** (worth 2.5 to 3.5 marks), so learn three:

- ▶ **The diamond problem (ambiguity):** if two parents both provide a method with the same name, the child does not know which to inherit.
- ▶ **Increased complexity:** the class hierarchy is harder to understand, design and maintain.
- ▶ **Name clashes:** attributes or methods with the same name in different parents collide, and resolving them is error-prone.

This is why several modern languages (such as Java) forbid multiple inheritance of classes and offer interfaces instead.

1. Define encapsulation and give an example.
2. What is polymorphism?
3. State two drawbacks of multiple inheritance.

ANSWERS

1. Bundling data with the methods that use it and hiding the data behind them; a BankAccount hides its balance and exposes deposit() and withdraw().
2. The same method name behaving differently on different classes (draw() on a Circle versus a Square).
3. Any two of: the diamond problem (ambiguity), increased complexity, name clashes.

MODULE FIVE · UNIT 3

Activity and Sequence Diagrams

By the end of this unit you can draw an activity diagram of a process flow and a sequence diagram of an interaction over time, the two behavioural diagrams the paper asks for by name.

An **activity diagram** is a flowchart of a process: it shows the order in which actions happen, including choices and parallel work. Its symbols:

- ▶ A **filled circle** for the start, and a filled circle in a ring for the end.
- ▶ A **rounded rectangle** for each action ("Check patient record").
- ▶ A **diamond** for a decision, with the condition on each outgoing arrow ("current patient?" splitting into yes and no).
- ▶ A **solid bar** (fork and join) where flows split to run in parallel and merge again.
- ▶ Arrows for the flow of control from one action to the next.

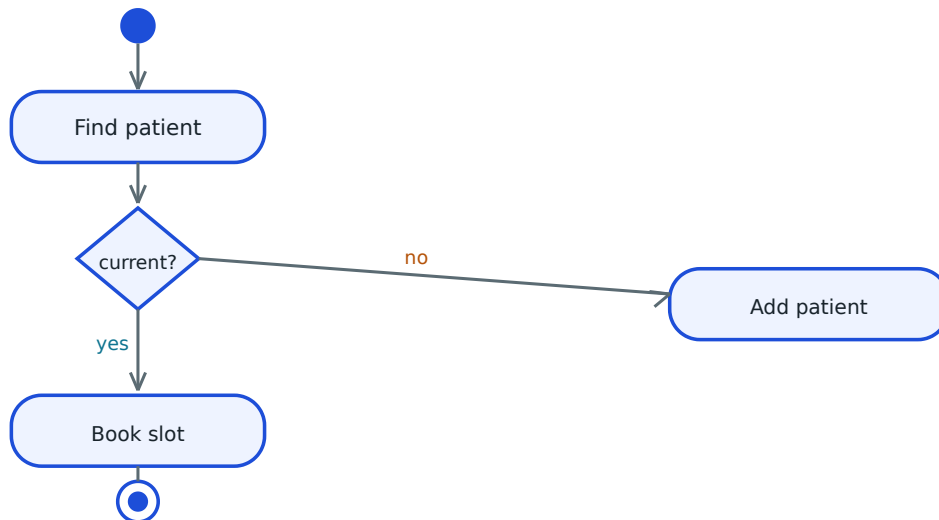


Figure 5.3 · An activity diagram. Filled circle starts it, rounded rectangles are actions, the diamond is a decision with a labelled branch on each arrow, and the ringed circle ends it.

TEACH

THE SEQUENCE DIAGRAM

A **sequence diagram** shows how objects interact *over time*, message by message. Time runs down the page. Its parts:

- ▶ A box at the top for each participating object or actor, with a dashed **lifeline** hanging down from it.
- ▶ A thin **activation bar** on a lifeline while that object is doing something.
- ▶ A solid arrow for a **message** (a call) from one lifeline to another, labelled with the operation; a dashed arrow for the **return**.

You draw one when a question says "draw a sequence diagram for the interaction", for example a sales representative entering an order of up to twelve products: the messages go from the rep to the system and back, one per product.

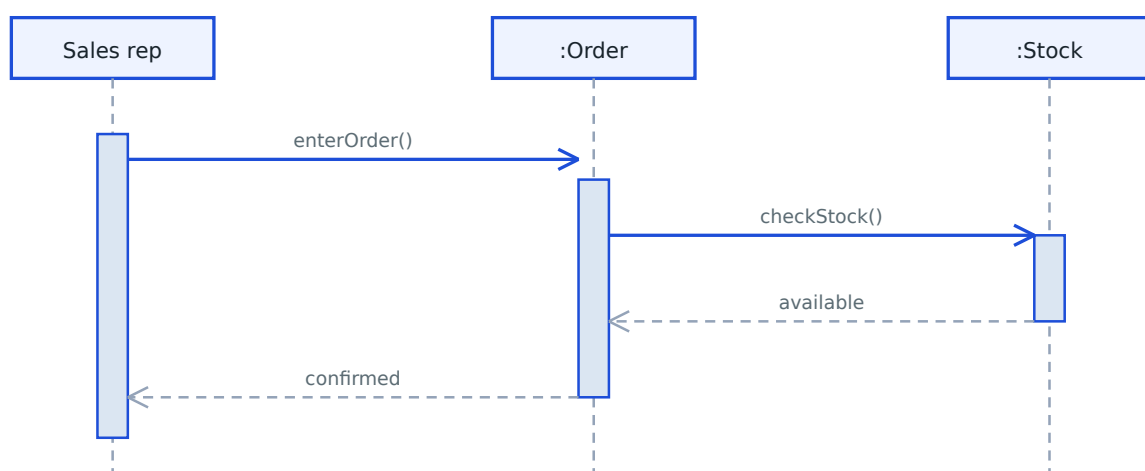


Figure 5.4 · A sequence diagram. Lifelines hang from each object, time runs downward, solid arrows are calls, dashed arrows are returns, and the thin bars show when each object is active.

- 1. What does a diamond mean in an activity diagram?**
 - 2. In a sequence diagram, which way does time run, and what is a lifeline?**
 - 3. Is a sequence diagram structural or behavioural?**
-

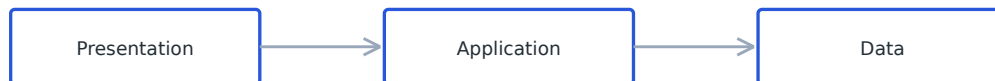
ANSWERS

1. A decision, with a condition labelling each outgoing branch.
2. Time runs down the page; a lifeline is the dashed vertical line showing an object's existence through time.
3. Behavioural (it shows behaviour over time).

MODULE SIX

Design, Architecture and Delivery

The Design and Implementation phases in the detail the paper asks for: how a system is structured across tiers, how you decide whether to build or buy it, how it is installed and supported, and how the project around it is managed. These are the marks in Section A and the last parts of the Section B scenario questions.



System Design and Architecture

By the end of this unit you can describe the client-server tiers, name the functions software architecture must provide and the non-functional requirements that shape hardware, state interface design principles, choose a system acquisition strategy, and design and populate a small relational database.

TEACH

CLIENT-SERVER TIERS

NOT IN THE COURSE TEXT A system's work can be split into three logical parts: **presentation** (the screens the user sees), **application** logic (the processing and rules), and **data** (storage and access). How these are spread across machines gives the **client-server tiers**, and the paper asks you to "explain the various client-server tiers to convince your boss":

- ▶ **Single-tier:** all three parts on one machine (a mainframe or a standalone app). Simple, but does not scale.
- ▶ **Two-tier:** a client holds the presentation (and often the application logic) and talks to a server that holds the data. The classic client and database server.
- ▶ **Three-tier:** presentation on the client, application logic on a separate middle (application) server, and data on a database server. Each tier can be scaled and maintained on its own; this is the common web architecture.
- ▶ **N-tier (multi-tier):** the middle tier is split further into more specialised servers. Most scalable, most complex.

Sell the three-tier to the boss on scalability and maintainability: because the tiers are separate, you can grow or replace one without disturbing the others.

TEACH

SOFTWARE ARCHITECTURAL FUNCTIONS

"Mention the software architectural functions the system will need to perform properly." Software architecture must provide four logical functions, which map onto the tiers above:

- ▶ **Data storage:** holding the data (the database).
- ▶ **Data access logic:** the processing needed to read and write that data (the queries).
- ▶ **Application logic:** the business rules and processing.
- ▶ **Presentation logic:** displaying information to the user and accepting their input.

"The choice of hardware architecture depends on four non-functional requirements; the first is operational, name two others." The four categories of non-functional requirement that shape the hardware are:

- ▶ **Operational requirements:** the environment the system must run in (operating system, network, hardware it must coexist with).
- ▶ **Performance requirements:** speed, capacity, throughput and availability.
- ▶ **Security requirements:** access control, encryption, protection of data.
- ▶ **Cultural and political requirements:** local laws, languages, customs and standards the system must respect.

Memory hook: O P S C, "Operational, Performance, Security, Cultural". These are the non-functional "-ilities" of Module Two, grouped for choosing hardware.

Interface design decides how the system interacts with the outside world, chiefly the graphical user interface (GUI) the user works through. "Outline three principles of interface design":

- ▶ **Consistency:** the same action looks and works the same way everywhere in the system, so the user can transfer what they learn.
- ▶ **Simplicity (and clarity):** show only what is needed, in plain layout and language, so the interface is easy to learn and use.
- ▶ **User feedback:** the system tells the user what is happening, confirms actions, and reports errors clearly.

Other acceptable principles: minimise user effort, prevent and handle errors gracefully, and respect the user's control.

When a system is needed but the team lacks the time or skill to build it, you choose a **system acquisition strategy**. The three options, and the one to pick under time pressure:

- ▶ **Build (custom development):** write it in-house. Best fit, most control, but slowest and needs the skills.
- ▶ **Buy (a packaged or off-the-shelf solution):** purchase ready-made software. Fast and cheaper, but may not fit exactly.
- ▶ **Outsource:** pay another company to build or run it. Good when you lack the skill or capacity in-house.

When a scenario says "the time frame is short and the project skills are not strategic", the answer is to **buy a packaged solution** or **outsource**, not to build. Supporting factors to cite: cost, time available, in-house skills, how well packages fit the need, and how strategic (core to the business) the system is.

The paper asks you to "design and populate a relational database of Customer, Order and Payment, with primary keys, at least three columns and five rows." A relational database stores data in **tables**; each table has a **primary key (PK)** column that uniquely identifies a row, and a **foreign key (FK)** to link to another table.

CUSTOMER

CUSTID (PK)	NAME	PHONE
C01	Ada Obi	0803...
C02	Bola Ade	0806...
C03	Chidi Eze	0701...
C04	Dele Ojo	0809...
C05	Efe Uche	0812...

ORDER

ORDERID (PK)	CUSTID (FK)	TOTAL
O01	C01	15000
O02	C03	8000
O03	C01	22000
O04	C05	5000
O05	C02	13000

The Payment table follows the same shape: payId (PK), orderId (FK), amount, with five rows. Notice custId is the primary key of Customer and a foreign key in Order: that FK is what records which customer placed each order.

Now you try: add the Payment table with five rows, and say which column links it back to the Order table.

RECALL

COVER THE ANSWERS

1. In a three-tier architecture, what sits in each tier?
2. Name the four non-functional requirements that shape hardware choice.
3. Time is short and the skills are not strategic. Which acquisition strategy?

ANSWERS

1. Presentation on the client, application logic on a middle server, data on a database server.
2. Operational, performance, security, cultural/political.
3. Buy a packaged solution, or outsource; do not build in-house.

Project Management, Installation and Support

By the end of this unit you can list the project planning activities, decide whether to accept new features, give the reasons projects succeed or fail, name the stakeholders and their views, and describe installation and support.

● MUST MEMORISE

PROJECT PLANNING ACTIVITIES

"Identify the five activities in project planning" (the Planning phase of Module One, as a delivery task):

1. Define the problem (the business problem and scope).
2. Produce the project schedule.
3. Confirm project feasibility (technical, economic, operational, schedule, legal).
4. Staff the project (allocate funds, assign members, obtain resources).
5. Launch the project.

TEACH

SCOPE CREEP AND FEATURE CREEP

Near the end of a project a client often asks for new features. **Scope creep** is uncontrolled growth of what the project must deliver; **feature creep** is the same at the level of individual features. The paper asks: as project manager, do you agree or refuse to add new features late? The exam-safe answer is to **refuse (or defer)**, and give reasons:

- ▶ It threatens the schedule and budget already agreed.
- ▶ It risks the quality and stability of what is nearly finished.
- ▶ New features belong in a change request or a later release, assessed properly, not bolted on under time pressure.

This is the "do not be afraid to cancel or revise scope" principle from Module One in action.

TEACH

WHY PROJECTS SUCCEED OR FAIL

"Enumerate reasons a project will succeed and reasons it can fail." The 25/26 paper gives you the first of each: a system succeeds when it is **acceptable to the customer** and fails if it **cannot establish upper-management commitment**. Others:

- ▶ **Succeed:** clear requirements, strong user involvement, realistic schedule and budget, good project management, management support.
- ▶ **Fail:** unclear or changing requirements, lack of user involvement, poor planning and estimation, weak management support, inadequate testing.

A **stakeholder** is anyone with an interest in the system. "Name three system stakeholders and describe how each views the system" wants you to show that different people want different things:

- ▶ **Users** want the system easy to use and helpful to their daily work.
- ▶ **Managers** want it to deliver business value on time and within budget.
- ▶ **Developers** (the technical team) want clear requirements and a system that is feasible to build and maintain.

Others: owners, customers, and regulators. The point the examiner wants is that these views can conflict, and the analyst must balance them.

Three loose ends the paper picks up:

- ▶ **Other job roles** an analyst can grow into: project manager, software architect, software tester, product designer, business analyst.
- ▶ **Installation (conversion)** uses one of the four strategies from Module One: direct cutover, parallel, phased, pilot. Choose by weighing risk against cost.
- ▶ **Support activities** (the Support phase): **maintain** the system (fix bugs, minor adjustments), **enhance** it (add capability as the business changes), and **support the users**, which includes running a help desk. "What three activities will the support group participate in?" is exactly these three.

Finally, the paper asks how the analyst explains to stakeholders what a **model** is, given that "a model is a representation of an important aspect of the real world." Explain it plainly: a model is a simplified picture of the system that leaves out detail so everyone can see and agree the important parts before it is built, the way an architect's drawing lets a client approve a building before a brick is laid. The diagrams throughout this manual, the use case, the DFD, the ERD, the class diagram, are all models in exactly this sense.

- 1. List the five project planning activities.**
 - 2. A client asks for new features near the deadline. What do you do, and why?**
 - 3. Name the three support-phase activities.**
-

ANSWERS

1. Define the problem, produce the schedule, confirm feasibility, staff the project, launch it.
2. Refuse or defer them, because they threaten the agreed schedule, budget and quality; handle them as a change request or later release.
3. Maintain the system, enhance the system, support the users (help desk).

REFERENCE

The Notation Legend and Must-Memorise List

Everything the examiner expects you to know cold, on a few pages, for the last hour before the hall. If you can reproduce the symbols on the next page and recite the lists after it from memory, you can answer any question on the paper.



The Notation Legend

Every symbol in the course on one page. Cover the right column and name each symbol; cover the left and draw each one.

● MUST MEMORISE

EVERY SYMBOL, ONE PAGE

DIAGRAM	SYMBOL	MEANS
Use case	stick figure	actor (a role outside the system)
Use case	oval	a use case (a verb phrase)
Use case	solid line	association (actor takes part)
Use case	dashed arrow, include	always uses (points to the included case)
Use case	dashed arrow, extend	optionally adds (points to the base case)
DFD	rectangle	external entity
DFD	numbered circle	process (a verb)
DFD	open-ended rectangle	data store (a noun, D1)
DFD	arrow	data flow (the data)
ERD	rectangle	entity
ERD	bar on line	one
ERD	circle on line	zero (optional)
ERD	crow's foot	many
ERD (Chen)	diamond	relationship
Class	three-part box	class: name / attributes / methods
Class	minus / plus	private / public visibility
Class	hollow diamond	aggregation (loose has-a)
Class	filled diamond	composition (life-or-death has-a)
Class	hollow triangle	generalization (is-a)
Activity	filled circle / ringed circle	start / end
Activity	rounded rectangle	action
Activity	diamond	decision (condition on each branch)
Sequence	dashed vertical line	lifeline (time runs down)
Sequence	solid / dashed arrow	message (call) / return

Four pairs are almost identical and each is a lost mark if confused. Fix them now:

- ▶ **Hollow vs filled diamond** (class): hollow is aggregation (part survives), filled is composition (part dies with the whole).
- ▶ **Include vs extend arrow** (use case): include points to the step always run; extend points to the base it might add to.
- ▶ **Bar vs crow's foot** (ERD): bar is one, crow's foot is many; the circle beside either makes it optional.
- ▶ **Closed box vs open-ended rectangle**: a closed rectangle is an entity or an external entity; the data store is open at the right.

REFERENCE • TWO

The Must-Memorise List

The lists the examiner asks you to reproduce. Recite each one from memory, then check. If you know these, Section A is done and Section B has no surprises.

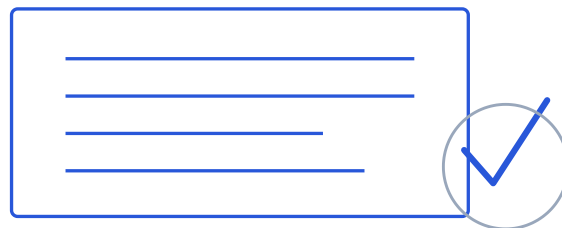
- ▶ **SDLC phases (PADIS):** Planning, Analysis, Design, Implementation, Support.
- ▶ **Analyst's skills:** technical, business, people (people is the most important).
- ▶ **Problem-solving sequence:** understand, verify benefit > cost, define requirements, alternatives, recommend, detail, implement, monitor.
- ▶ **Feasibility (TEOSL):** Technical, Economic, Operational, Schedule, Legal.
- ▶ **Elicitation techniques:** interview, questionnaire, observation, document review, JAD, prototyping.
- ▶ **Requirement types:** functional (what it does) vs non-functional (a quality); and business / user / system levels.
- ▶ **Use case relationships:** association, include, extend, generalization.
- ▶ **DFD elements:** external entity, process, data store, data flow.
- ▶ **DFD mechanical errors:** black hole (no output), miracle (no input), grey hole (too little input).
- ▶ **Cardinal relationships:** one-to-one, one-to-many, many-to-many.
- ▶ **Object-oriented concepts:** encapsulation, inheritance, polymorphism, composition, aggregation, association.
- ▶ **Multiple inheritance drawbacks:** the diamond problem, complexity, name clashes.
- ▶ **UML categories:** structural (class, object) and behavioural (use case, activity, sequence). Inventors: Booch, Rumbaugh, Jacobson.
- ▶ **Client-server tiers:** single, two, three, n-tier.
- ▶ **Hardware non-functional requirements (OPSC):** operational, performance, security, cultural/political.
- ▶ **Acquisition strategies:** build, buy, outsource.
- ▶ **Conversion strategies:** direct cutover, parallel, phased, pilot.
- ▶ **Support activities:** maintain, enhance, support the users (help desk).

- ▶ **Analyst:** a business problem solver, the bridge between users and the technical team.
- ▶ **Model vs tool vs technique vs methodology:** a model represents the system; a tool is software you use; a technique is a way of doing one activity; a methodology is the whole package.
- ▶ **Predictive vs adaptive:** predictive plans everything upfront; adaptive plans as it goes in short iterations.
- ▶ **Cardinality vs ordinality:** cardinality is the maximum (one or many); ordinality is the minimum (mandatory or optional).
- ▶ **Object vs class:** a class is the blueprint; an object is an instance of it.
- ▶ **Composition vs aggregation:** composition the part dies with the whole; aggregation the part lives on.
- ▶ **Data model vs process model:** a data model (ERD) shows data at rest; a process model (DFD) shows data in motion.
- ▶ **Forward vs backward chaining:** read a relationship in the direction of the verb, then the reverse.

WORKED PAPER 2025/2026

The 2025/2026 Paper, Fully Worked

The most recent paper, the one your sitting will most resemble, quoted exactly as it was printed and then answered in full. Each question appears first in an As-printed box in the examiner's own words, then a Break-it-down box reads it in your shoes, then the answer. Cover the answer and attempt each question before you read on.



Section A: Questions One and Two

You answer one of these two in the hall. Both are worth working: they drill the ERD, the DFD, the class, use case and activity diagrams in one sitting.

AS PRINTED

PQ 25/26 Q1

A landlord owns one of many houses and asks the only estate agent he knows to help find a suitable tenant. The agent (who manages many houses) contacts his supervisor to inform him about the availability of the house. The supervisor indicates interest and agrees to rent the house.

After the agreement, the supervisor moves into the house with the help of the agent. Before occupation, they both clean the house. Once settled, the supervisor occupies the house and pays rent through the agent, who then forwards the payment to the landlord.

- a. Draw an ERD to model the relationships between the identified entities (4.5 mks)
- b. Data Flow Diagram (DFD):
 - i. Draw a Context diagram to model the entire system (1.5 mks)
 - ii. Draw a Level 0 DFD to show various processes involved (4.5 mks)
 - iii. Draw a Level 1 DFD to model the process of rent payment (3 mks)
- c. The resulting system is successful when it is acceptable to the customer and it will fail if it cannot establish upper-management commitment to the project. Enumerate (3) other reasons a project will succeed and (3) other reasons it can fail (3 mks)
- d. Read and interpret the following (using forward and backward chaining) (1 mk)

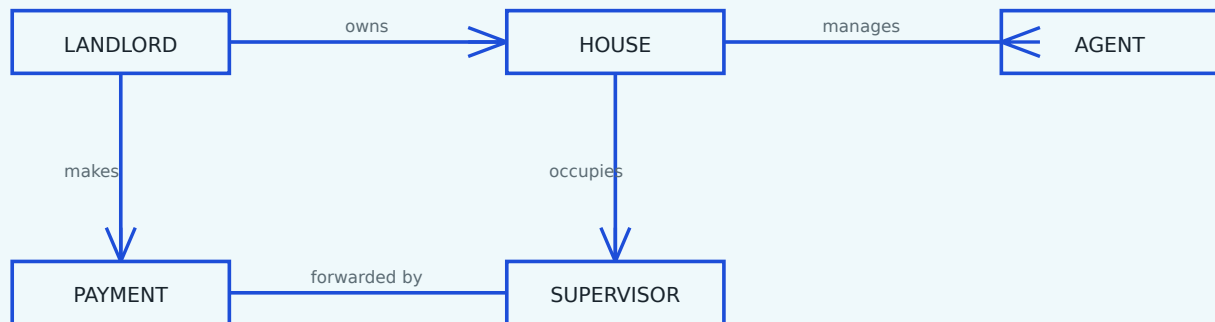
Under part d the paper prints a small crow's-foot fragment: a **Customer** box joined to an **Order** box, with a single bar at the Customer end and a circle-and-crow's-foot at the Order end. It is reproduced as our own figure in the answer to d.

BREAK IT DOWN

- GIVEN** A landlord, an estate agent, a supervisor (the tenant), houses, and rent payments that flow supervisor to agent to landlord.
- ASKED** Four separate things: an ERD (a), three DFDs (b), reasons a project succeeds and fails (c), and reading a printed ERD fragment (d). Parts a, b and d test Modules Three and Four; part c tests Module Six.
- MARKS** 17.5 in all. Part b is worth the most (9), so the three DFDs deserve the most time and care.
- WATCH** The examiner asks the ERD in no particular notation, so use the crow's foot. In the DFDs, keep every flow touching a process, and make the three levels balance.

A. ENTITY-RELATIONSHIP DIAGRAM (4.5 MARKS)

The nouns kept as records are LANDLORD, HOUSE, AGENT, SUPERVISOR (the tenant) and PAYMENT. The relationships from the story: a landlord *owns* many houses; an agent *manages* many houses; an agent *serves* a supervisor; a supervisor *occupies* a house; a supervisor *makes* payments; an agent *forwards* payments to the landlord.



One LANDLORD owns many HOUSEs; one AGENT manages many HOUSEs; one HOUSE is occupied by one SUPERVISOR; one SUPERVISOR makes many PAYMENTs, each forwarded by the AGENT to the LANDLORD.

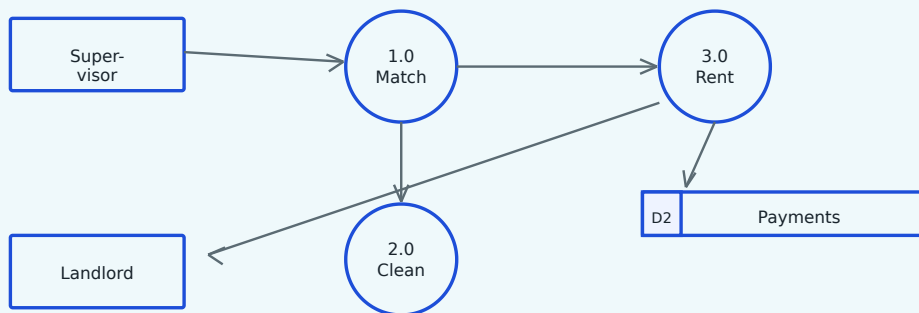
ANSWER, CONTINUED

B. DATA FLOW DIAGRAMS (9 MARKS)

Context diagram: one process "0 Rental system" in the middle, with SUPERVISOR and LANDLORD as external entities. Flows: supervisor sends "house request" and "rent payment" in; the system sends "house availability" back to the supervisor and "forwarded payment" to the landlord. No data stores at this level (draw it exactly as Figure 3.2 in Module Three, with these labels).

Level 0: explode process 0 into the major processes: 1.0 Match tenant to house, 2.0 Agree and clean, 3.0 Collect and forward rent. Add data stores D1 Houses and D2 Payments. Wire the flows so each process has an input and an output, and so the "house request" in and "forwarded payment" out still match the context diagram (balance).

Level 1 of process 3.0 (rent payment): explode 3.0 into 3.1 Receive payment from supervisor, 3.2 Record payment (to D2 Payments), 3.3 Forward payment to landlord. The total flows in and out (payment in, forwarded payment out) balance process 3.0.



The rental Level 0 DFD. Process 0 has exploded into 1.0, 2.0 and 3.0; the rent flows through 3.0 to the landlord and into D2 Payments; every flow touches a process, and the diagram balances the context.

ANSWER, CONTINUED

C. THREE MORE REASONS TO SUCCEED AND THREE TO FAIL (3 MARKS)

Succeed (besides customer acceptance): clear and agreed requirements; strong user involvement throughout; a realistic schedule and budget with good project management.

Fail (besides lack of upper-management commitment): unclear or constantly changing requirements; little or no user involvement; poor planning, estimation or testing.

D. READING THE FRAGMENT (1 MARK)

The bar at the Customer end means "one" and the circle-with-crow's-foot at the Order end means "zero or many". So, **forward chaining:** one CUSTOMER places zero or many ORDERS. **Backward chaining:** each ORDER is placed by one and only one CUSTOMER.

A patient visits a hospital to use the patient billing system in order to make enquiries about the types of drugs he can buy. He must have insurance or he won't be attended to by the Nurse. The nurse checks his vitals and refers him to a doctor who in turn refers him to the Insurance Carrier to confirm the patient has his insurance covered. On confirmation, the doctor advises the patient on which drugs to use for his condition.

Note that a patient must be assigned to only one doctor but a doctor can have many patients. A patient can have only one insurance ID to prevent insurance fraud.

- a. Draw a class diagram for this scenario. [Hint: Show multiplicity where necessary] (5 mks)
- b. Draw a use case diagram for the system. (5 mks)
- c. Draw an activity diagram to show the system's process flow. (5 mks)
- d. State three (3) drawbacks of multiple inheritance. (2.5 mks)

BREAK IT DOWN

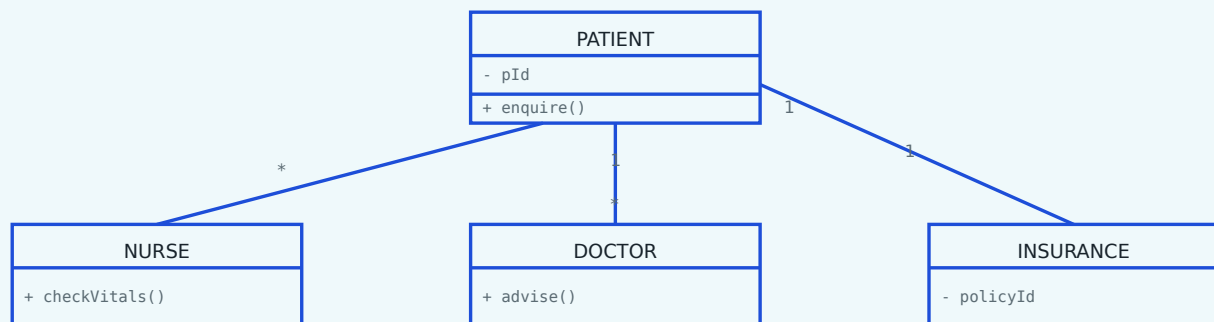
GIVEN A patient billing system with a Patient, a Nurse, a Doctor and an Insurance Carrier, and two firm rules: one doctor per patient (a doctor has many), one insurance ID per patient.

ASKED Three diagrams (class, use case, activity) and one written part (drawbacks of multiple inheritance). All four come straight from Module Five.

WATCH The class diagram must show the multiplicities the rules give you (Patient 1 to Doctor's many; Patient 1 to Insurance 1). Do not confuse the three diagrams: the class diagram is structural, the other two behavioural.

A. CLASS DIAGRAM (5 MARKS)

Four classes: PATIENT, NURSE, DOCTOR, INSURANCE. Give each attributes and a method. The rules give the multiplicities: a DOCTOR treats many PATIENTs but each PATIENT has one DOCTOR (1 to *); a PATIENT has one INSURANCE and an insurance covers one patient (1 to 1); a NURSE attends many PATIENTs (1 to *).



PATIENT to DOCTOR is many-to-one (a doctor has many patients, a patient one doctor); PATIENT to INSURANCE is one-to-one; NURSE attends many patients.

B. USE CASE DIAGRAM (5 MARKS)

Actors: PATIENT, NURSE, DOCTOR, INSURANCE CARRIER. Use cases inside the boundary: Make enquiry, Check vitals, Refer to doctor, Confirm insurance, Advise on drugs. "Attend patient" **includes** "Check insurance" (it must always happen, since he will not be attended without insurance). Draw it as Figure 2.1 in Module Two with these labels.

C. ACTIVITY DIAGRAM (5 MARKS)

Start, then action "Patient makes enquiry", then a decision "has insurance?": the "no" branch ends (not attended); the "yes" branch flows to "Nurse checks vitals", "Refer to doctor", "Doctor confirms insurance with carrier", "Advise on drugs", then end. Draw it as Figure 5.3 with this flow.

D. DRAWBACKS OF MULTIPLE INHERITANCE (2.5 MARKS)

The diamond problem (ambiguity when two parents give the same method); increased complexity of the class hierarchy; and name clashes between members inherited from different parents.

WORKED PAPER 2025/2026**Section B: Questions Three to Six**

You answer three of these four. Q3 and Q5 are written questions; Q4 mixes architecture with a database design; Q6 is requirements and UML. Practise all four.

Tikki Tikka Buttry and Pastries need to automate their food selling business in order to meet up with the demands of operating in Nigeria. They hired Billy Butcher as their analyst to guide them on the steps to take in order to make their automation a success, they also hired a graphic designer and a JavaScript programmer to join the team; they call themselves the Boys.

- a. Briefly explain the THREE questions that the Boys will need to answer during the Feasibility Analysis phase of their system planning (3 mks)
- b. Billy's first approach to problem solving is Research and Understand the Problem. Outline any other six (6) steps he can take (3 mks)
- c. A model is a representation of an important aspect of the real world. How can the Boys properly explain what a model is to the stakeholders? (4 mks)
- d. The most important skill Billy can have is People knowledge and skill; explain in detail any other skill he can possess to do a great job (3 mks)
- e. Billy needs to ensure there is adequate support for the users of the new system. What three activities will the support group need to participate in? (3 mks)
- f. Mention any three other job roles that Billy can fit into (1.5 mks)

BREAK IT DOWN

GIVEN A food business automating, with an analyst (Billy) and a small team.

ASKED Six short written parts, all straight recall from Modules One and Six: feasibility, the problem-solving steps, what a model is, the analyst's skills, support activities and job roles.

WATCH Each part names the exact list it wants. Answer the number asked (three questions, six steps, three activities, three roles) and no more.

A. THREE FEASIBILITY QUESTIONS (3 MARKS)

Feasibility analysis asks whether the project is worth pursuing, from three angles: **Is it technically feasible?** (do we have the technology and skill to build it?); **Is it economically feasible?** (do the benefits outweigh the costs?); and **Is it operationally feasible?** (will the users accept and use it, and does it fit the business?). Schedule and legal feasibility complete the five, but these three are the core questions.

B. SIX MORE PROBLEM-SOLVING STEPS (3 MARKS)

After researching and understanding the problem: verify the benefits outweigh the costs; define the requirements; develop alternative solutions; recommend the best solution; define the details of the chosen solution; implement it (and then monitor the results).

C. EXPLAINING WHAT A MODEL IS (4 MARKS)

Tell the stakeholders a model is a simplified picture of the system that leaves out detail so everyone can see and agree the important parts before it is built, the way an architect's drawing lets a client approve a building before a brick is laid. The use case diagram, DFD and class diagram they will be shown are all such models.

D. ANOTHER SKILL (3 MARKS)

Besides people skills, Billy needs **technical knowledge and skills**: an understanding of computers, networks, databases and the tools and techniques of system building, so he can understand and communicate technical requirements even though the programmer writes the code. (Business knowledge is an equally acceptable answer.)

E. THREE SUPPORT ACTIVITIES (3 MARKS)

Maintain the system (fix bugs and make minor adjustments); enhance the system (add capability as the business grows); and support the users, which includes running a help desk.

F. THREE OTHER JOB ROLES (1.5 MARKS)

Project manager, software architect, and product designer (also acceptable: software tester, business analyst).

Vought Industries have decided to acquire a Pharmaceutical Information System; the time frame is short and the project skills are not strategic

a. What System Acquisition Strategy fits these requirements? Enumerate (3) other factors that support your choice. (3 mks)

b. Mention any two (2) software architectural functions that Vought Industries will need in order to make the system perform properly. (3 mks)

c. The choice of hardware architecture depends on four (4) nonfunctional requirements, the first is operational requirements, enumerate any other two types of hardware requirements. (3 mks)

d. Interface design is the process of defining how the system will interact with external entities. Outline any three (3) principles of Interface design. (3 mks)

e. Design and populate a Relational Database consisting of Customer, Order and Payment Type for the Pharmaceutical System (use dummy names and values and ensure to include primary keys for all unique IDs. You should have a minimum of 3 columns and 5 rows) (5.5 mks)

BREAK IT DOWN

- GIVEN** A firm buying a system, under time pressure, with no strategic in-house skills. Those two clues decide part a.
- ASKED** The acquisition strategy and four short parts from Module Six Unit 1, then a small database to design and populate.
- WATCH** "Short time frame" and "skills not strategic" both point away from building. Part e wants real dummy rows, not a description: draw the tables.

A. ACQUISITION STRATEGY (3 MARKS)

Because the time frame is short and the skills are not strategic, **buy a packaged (off-the-shelf) solution, or outsource**, rather than build in-house. Supporting factors: the time available (buying is fastest); the in-house skills (they are lacking); and how strategic the system is (it is not core, so a package is fine); also cost and how well available packages fit the need.

B. TWO SOFTWARE ARCHITECTURAL FUNCTIONS (3 MARKS)

Any two of: data storage, data access logic, application (business) logic, and presentation logic.

C. TWO MORE HARDWARE NON-FUNCTIONAL REQUIREMENTS (3 MARKS)

Besides operational requirements: performance requirements and security requirements (the fourth is cultural and political requirements).

D. THREE INTERFACE DESIGN PRINCIPLES (3 MARKS)

Consistency (the same action works the same way everywhere); simplicity and clarity (show only what is needed); and user feedback (the system confirms actions and reports errors clearly).

E. DESIGN AND POPULATE THE DATABASE (5.5 MARKS)

CUSTID (PK)	NAME	PHONE
C01	Ada Obi	0803...
C02	Bola Ade	0806...
C03	Chidi Eze	0701...
C04	Dele Ojo	0809...
C05	Efe Uche	0812...

ORDERID (PK)	CUSTID (FK)	DRUG
O01	C01	Paracetamol
O02	C03	Amoxicillin
O03	C01	Vitamin C
O04	C05	Ibuprofen
O05	C02	Metformin

PAYID (PK)	ORDERID (FK)	TYPE
P01	O01	Cash
P02	O02	Card
P03	O03	Insurance
P04	O04	Card
P05	O05	Transfer

Each table has a primary key; custId links Order back to Customer and orderId links Payment Type back to Order, both as foreign keys.

AS PRINTED

PQ 25/26 Q5

A federal teaching hospital intends to replace its fragmented patient appointment, clinical record, laboratory request, and medication tracking systems with a single integrated Clinical Workflow and Patient Safety Management System. The current system is partly paper-based and partly supported by disconnected departmental applications. Doctors complain that patient records are incomplete, nurses report delays in medication administration, laboratory scientists complain about duplicated test requests, and hospital administrators are concerned about regulatory compliance, audit trails, and data privacy.

The hospital's board wants the system delivered within twelve months. However, stakeholders disagree on system priorities. Doctors want speed and flexibility, nurses want ease of use, laboratory staff want accurate workflow integration, and administrators insist on strict documentation, traceability, and reliability. Failure in the new system could affect patient safety.

a. Briefly compare seven (7) methodologies that can be used in this project. (7 mks)

[Hint: your answer should be short and concisely state why the methodologies may be suitable or not]

b. Identify five (5) activities in project planning. (5 mks)

c. Using the case, formulate at least:

- i. two (2) business requirements,
- ii. two (2) user requirements,
- iii. three (3) functional requirements,
- iv. two (2) non-functional requirements, and
- v. two (2) system requirements, (5.5 mks)

BREAK IT DOWN

GIVEN A hospital replacing fragmented systems, safety critical, twelve-month deadline, stakeholders who disagree.

ASKED Compare seven methodologies, list five planning activities, and formulate requirements at every level. Modules One, Two and Six.

WATCH Part a wants a *comparison* with a reason each is suitable or not for this safety-critical project, not just a list. Part c wants the exact counts across five levels.

A. COMPARE SEVEN METHODOLOGIES (7 MARKS)

- ▶ **Waterfall:** strong documentation and traceability suit a safety-critical system, but its rigidity is risky if requirements shift.
- ▶ **Prototyping:** good for the unclear priorities here, but the prototype can be mistaken for the finished safety system.
- ▶ **RAD:** fast, which suits the twelve-month deadline, but weak for a large, complex, safety-critical hospital system.
- ▶ **Spiral:** its risk analysis each cycle fits a safety-critical project well, but it is heavy and slow.
- ▶ **Incremental:** delivers working pieces early, useful given the deadline, but integration across departments must be managed.
- ▶ **Agile:** handles the disagreeing stakeholders through constant feedback, but its light documentation clashes with the audit-trail requirement.
- ▶ **V-shape:** pairs each stage with its test, excellent for the reliability and traceability a hospital demands, but as rigid as waterfall.

ANSWER, CONTINUED**B. FIVE PROJECT PLANNING ACTIVITIES (5 MARKS)**

Define the problem; produce the project schedule; confirm feasibility; staff the project; and launch the project.

C. FORMULATE REQUIREMENTS (5.5 MARKS)

- ▶ **Business:** cut medication-administration delays; achieve regulatory compliance with full audit trails.
- ▶ **User:** a nurse can administer and record medication at the bedside; a doctor can see a complete patient record in one place.
- ▶ **Functional:** the system shall record each patient appointment; the system shall flag duplicated laboratory test requests; the system shall log every change to a record for audit.
- ▶ **Non-functional:** the system shall be available 99.9% of the time; the system shall encrypt all patient data (privacy).
- ▶ **System:** the system shall store records as time-stamped entries linked to a patient ID; the system shall integrate with the existing departmental applications.

A private university wants to build an AI-supported programming learning platform for undergraduate students. The system should help students learn Python, Java, and C++ by providing coding exercises, automated feedback, debugging support, lecturer dashboards, plagiarism alerts, and adaptive learning recommendations. The university expects the system to support 8,000 students across three campuses.

At the start of the project, requirements are unclear. Students want instant answers to coding problems, lecturers want the system to promote independent thinking rather than answer copying, administrators want analytics on student performance, and the ICT unit is concerned about security, scalability, integration with the learning management system, and maintenance cost. Halfway through the project, university management requests additional features: AI-generated lecture notes, automatic grading of essays, and integration with the university payment portal.

- a. Identify the weaknesses of the as-is system, the expected improvements in the to-be system, and the assumptions that must be validated before development begins. (3.5 mks).
- b. Design a requirements elicitation plan for the system using at least four techniques. For each technique, identify the stakeholders involved, the type of information to be collected, and why the technique is suitable for this project. (4 mks).
- c. Write short notes on: Unified Modeling Language (UML) (also state the inventors and its uses). (2 mks)
- d. Categorise UML diagrams into two broad categories. Mention two diagrams in each of the categories and describe its function (4 mks)
- e. With the aid of diagrams, explain the following class relationships in system analysis: (i) Inheritance, (ii) Composition, (iii) Aggregation, and (iv) Association. (4 mks)

The paper writes "Categories UML diagrams" for "Categorise", and "describe its function" for "its function"; both are the paper's own slips, kept here as printed. Do not copy them into your answer.

BREAK IT DOWN

- GIVEN** An AI learning platform, unclear requirements, and mid-project feature requests (the scope-creep trap in disguise).
- ASKED** As-is/to-be analysis, an elicitation plan, and three straight-recall UML parts. Modules Two and Five.
- WATCH** Part a wants three lists (weaknesses, improvements, assumptions). Part b wants four techniques each with stakeholder, information and justification, not just a list of technique names.

A. AS-IS, TO-BE AND ASSUMPTIONS (3.5 MARKS)

As-is weaknesses: no single platform, so learning is unsupported; no automated feedback or plagiarism checking; no analytics for administrators. **To-be improvements:** one platform teaching three languages with automated feedback, debugging support, plagiarism alerts and analytics. **Assumptions to validate:** that 8,000 students across three campuses have reliable network access; that the platform can integrate with the existing learning management system; that the AI feedback is accurate enough to be trusted.

B. ELICITATION PLAN (4 MARKS)

- ▶ **Interview** lecturers: information on the learning outcomes and marking rules; suitable because they hold the academic requirements and can be probed in depth.
- ▶ **Questionnaire** to students: information on what help they want; suitable because it cheaply reaches 8,000 scattered students.
- ▶ **Observation** of a lab session: information on how students really code and get stuck; suitable because it shows real behaviour.
- ▶ **Document review** of the learning management system: information on existing data and integration points; suitable because it grounds the design in what exists.

C. SHORT NOTES ON UML (2 MARKS)

UML is a standard set of diagram notations for modelling object-oriented systems, created by Booch, Rumbaugh and Jacobson. Its uses: to visualise, specify, construct and document a software system in one shared language.

D. TWO CATEGORIES OF UML DIAGRAM (4 MARKS)

Structural diagrams model what the system is: the class diagram (classes and relationships) and the object diagram (objects at a moment). **Behavioural** diagrams model what it does: the use case diagram (what users can do) and the activity diagram (a process flow); the sequence diagram is another.

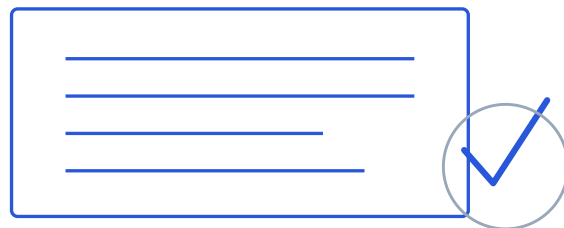
E. FOUR CLASS RELATIONSHIPS (4 MARKS)

Inheritance (hollow triangle, is-a): a Postgraduate is a Student. **Composition** (filled diamond, life-or-death has-a): an Order is made of Order Lines. **Aggregation** (hollow diamond, loose has-a): a Department has Lecturers who survive it. **Association** (plain line): a Doctor treats a Patient. Draw the four side by side as Figure 5.2 in Module Five.

WORKED PAPER 2024/2025

The 2024/2025 Paper, Fully Worked

The paper from the session before the target, in the same three-hour, one-from-A and three-from-B shape. Working it after the 2025/2026 paper shows you how stable the examiner's habits are: the same scenarios (patient appointments, sales management, the hospital ward) and the same diagram set come round again and again.



Section A: Questions One and Two

The two written Section A questions: requirements and methodology, then the life cycle and its stakeholders. Answer one in the hall; work both here.

AS PRINTED

PQ 24/25 Q1

As a systems analyst working on a new library management system, you are tasked with gathering system requirements from various stakeholders.

- a. Identify and briefly explain two appropriate requirement elicitation techniques you would use. For each technique, justify why it is suitable for this project and specify one advantage and one potential limitation. (4 Marks)
- b. Based on your selected elicitation techniques, outline:
 - i. Three functional requirements. (3 Marks)
 - ii. Three non-functional requirements. (3 Marks)
 - iii. List one reason why requirements elicitation should involve users from different organizational levels. (0.5 Mark)
- c. You are the chosen project manager for a new line of clothing about to be opened. You very much need the project to succeed and are willing to give it your all.
 - i. The requirements of this work MAY NOT be clearly stated at the beginning. Choose an appropriate System Development Lifecycle (SDLC) methodology and map out the phases of the project. (4 Marks)
 - ii. You are nearing the end of the project and the client is insisting on adding new features but you want to prevent scope and feature creep. Do you refuse or agree to add the new features? Give three cogent reasons for your decision as Project Manager. (3 Marks)

BREAK IT DOWN

ASKED Elicitation techniques with justification (a); requirements at two levels plus a reason (b); a methodology choice for unclear requirements (c i); and the scope-creep decision (c ii). Modules Two, One and Six.

WATCH Part a wants four things per technique: name, why it suits, one advantage, one limitation. Part c i's clue is "may not be clearly stated", which points straight at an adaptive method.

A. TWO ELICITATION TECHNIQUES (4 MARKS)

Interview the head librarian: suitable because they hold the borrowing rules and can be probed in depth; advantage, rich detailed information; limitation, slow and only a few people. **Questionnaire** to student borrowers: suitable because it cheaply reaches the large, scattered student body; advantage, wide reach at low cost; limitation, no follow-up and often a low response rate.

B. REQUIREMENTS AND A REASON (6.5 MARKS)

Functional: the system shall let a member borrow a book; shall let a member reserve a book; shall calculate and record fines. **Non-functional:** the system shall respond within two seconds; shall be available 99.9% of the time; shall keep member data secure. **Reason for multiple levels:** different levels (a student, a librarian, a head of library) use the system differently, so gathering from all of them captures needs that a single level would miss.

C. METHODOLOGY AND SCOPE CREEP (7 MARKS)

i. Unclear requirements call for an adaptive method: choose **prototyping** or **RAD**, which discover requirements as they go. Map the phases: plan, then build a prototype, review it with the client, refine, and repeat until the requirements settle, then implement and support. **ii. Refuse (or defer)** the new features, because they threaten the agreed schedule and budget, they risk the quality of the nearly finished system, and they should be handled as a change request or a later release.

- a. You are to train a team of young professional on System Development Lifecycle (SDLC) and have chosen a four phased cycle namely; analysis, design, implementation and support. State the objectives of these phases and the activities involved in each. (8 marks)
- b. Name three system stakeholders and describe how each one views an information system. (3 marks)
- c. i) Differentiate between tools and techniques with respect to system development. (2 marks) ii) Give two examples of tools and techniques. (2 marks)
- d. XYZ Conglomerate has contracted you to develop a new information system for its administrative processes. Outline the key information you would need about the business in order to design a system that effectively meets its needs. (2.5 marks)

BREAK IT DOWN

WATCH Part a names four phases (not five): the four after planning. For each give an objective and its activities. Parts b, c, d are short recall from Modules One and Six.

A. FOUR PHASES: OBJECTIVES AND ACTIVITIES (8 MARKS)

Analysis: objective, understand and document what the system must do; activities, gather information, define and prioritise requirements, build prototypes, evaluate alternatives. **Design:** objective, specify how it will be built; activities, design the architecture, network, interfaces, database and controls. **Implementation:** objective, a reliable working system with trained users; activities, construct, test, convert data, train and document, install. **Support:** objective, keep it running for years; activities, maintain, enhance, support the users (help desk).

B. THREE STAKEHOLDERS (3 MARKS)

Users want it easy and helpful for daily work; managers want business value on time and on budget; developers want clear requirements and a feasible, maintainable system.

C. TOOLS VERSUS TECHNIQUES (4 MARKS)

A tool is a software product used to do the work; a technique is a way of completing one activity. Tools: a modelling package, an IDE. Techniques: interviewing, cost-benefit analysis.

D. KEY BUSINESS INFORMATION FOR XYZ (2.5 MARKS)

The business goals and processes, the people and their roles, the data the business keeps, the existing systems and their weaknesses, and the rules and constraints the new system must respect.

WORKED PAPER 2024/2025

Section B: Questions Three to Six

The four diagram questions. Q3 and Q4 are ERD and DFD heavy; Q5 is class diagrams and object-oriented concepts; Q6 is the analyst plus a use case and activity.

- a. What are the elements of a Data Flow diagram? Enumerate any four (4) you know. (2 marks)
- b. Define what is meant by a data store in a process model. How should a data store be named? (1.5 marks)
- c. Read and interpret the Entity Relationship Diagram (ERD) presented in Figure 1 by providing the cardinal relationships between the entities (3 marks)
- d. A warehouse has recently enforced a new management system to streamline the receiving process of stock. When truck drivers arrive, they are required to submit their trucks to the warehouse. On submission, the warehouse manager instructs the manual laborer to unload the trucks. It is important to note that a truck may or may not be carrying any stock. Once unloading is complete, the storekeeper is responsible for taking inventory of all available stock (spoilt, missing and in-good condition). The storekeeper then generates a comprehensive stock report, which is disseminated to the warehouse manager, the laborer involved in the unloading and the truck driver. Separately, both the truck drivers (regarding their delivery completion) and the labourers (regarding their unloading tasks) are required to submit individual activity reports directly to the manager.
 - i. Draw an ERD to model the relationships between all identified entities. (5 marks)
 - ii. Draw a Level 0 DFD to model the various processes involved. (4.5 marks)
- e. After drawing the Data flow diagrams for the above scenario, outline three (3) ways you can validate the integrity and correctness of the diagrams (1.5 marks)

Part c refers to a printed Figure 1: a large crow's-foot ERD for a music service (entities CUSTOMER, SALE, TARGETED PROMOTION, CUSTOMER FAVORITE, CUSTOMER INTEREST and AVAILABLE TUNE, with relationships such as makes, is targeted by, adds, creates, includes and promotes). The answer reads its cardinalities below.

BREAK IT DOWN

ASKED Two recall parts (a, b), a diagram-reading part (c), a full ERD plus Level 0 DFD from the warehouse story (d), and diagram validation (e). Modules Three and Four.

WATCH In d, the entities are TRUCK DRIVER, TRUCK, WAREHOUSE MANAGER, LABORER, STOREKEEPER, STOCK, STOCK REPORT, ACTIVITY REPORT. "A truck may or may not be carrying stock" is an optional relationship (zero or many).

A. DFD ELEMENTS (2 MARKS)

External entity, process, data store, data flow.

B. DATA STORE (1.5 MARKS)

A data store is data at rest in the system, a file or table where data is held between processes. It is named with a **noun** (for example D1 Stock), never a verb, and given a store number.

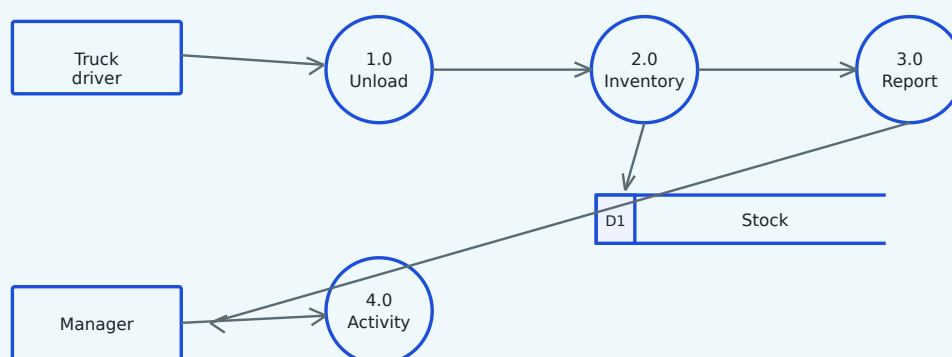
C. READING FIGURE 1 (3 MARKS)

Read each relationship both ways from its crow's-foot marks. For example: one CUSTOMER makes many SALES, and each SALE is made by one CUSTOMER (1:M); one CUSTOMER is targeted by many TARGETED PROMOTIONS, and each promotion targets many customers (M:N); a CUSTOMER adds many CUSTOMER FAVORITES (1:M); an AVAILABLE TUNE is included in many CUSTOMER INTERESTS (M:N). State the cardinality (one or many, optional or mandatory) for each line the figure draws.

ANSWER, CONTINUED**D. WAREHOUSE ERD AND LEVEL 0 DFD (9.5 MARKS)**

ERD: entities TRUCK DRIVER, TRUCK, MANAGER, LABORER, STOREKEEPER, STOCK, STOCK REPORT, ACTIVITY REPORT. A TRUCK DRIVER submits many TRUCKS; a TRUCK may carry zero or many STOCK (optional); a LABORER unloads many TRUCKS; a STOREKEEPER takes inventory of STOCK and generates STOCK REPORTS; drivers and labourers submit ACTIVITY REPORTS to the MANAGER. Crow's foot throughout, with the optional (circle) mark on the truck-to-stock line.

Level 0 DFD: external entities TRUCK DRIVER and MANAGER; processes 1.0 Receive and unload truck, 2.0 Take inventory, 3.0 Generate stock report, 4.0 Submit activity report; data stores D1 Stock and D2 Reports. Wire the flows so each process has input and output, and balance them against the context diagram.



The warehouse Level 0 DFD. Each process has an input and an output, no flow skips a process, and the stock report reaches the manager, driver and labourer.

E. VALIDATE THE DIAGRAMS (1.5 MARKS)

Check every process has both an input and an output (no black holes or miracles); check no illegal flow (every flow touches a process); and check the levels balance (the flows in and out of a process match its explosion).

- a. As a system analyst, you have been asked to draw a Data Flow diagram to explain the processes involved in food ordering at a Nigerian airport. Outline any four (4) reasons it is helpful to first Model the processes. (2 marks)
- b. i) Using appropriate diagrammatic illustrations, explain the various cardinal relationship in ERD (use Information Engineering style) (3 marks)
- ii) What makes a relationship optional? With a diagram, give a practical illustration. (1.5 marks)
- iii) What does the term 'Data Balancing' mean? (1 mark)
- c. In a tabular form, outline three major differences between Process Modelling and Data Modelling (1.5 marks)
- d. You are to develop a sales management system that allows sales representative to record orders placed by customers, checkout orders, and also allows the store keeper to keep inventory of product in the warehouse. A customer can place more than one order however each order must be placed by one customer. Each sales representative serves zero to many customers per time, however each customer is served by only one sales representative. Each order contains one or more products and zero or more products are stores in only one warehouse. Draw an entity relationship diagram for this scenario. (3 marks)
- e. 'Data cannot move between entities in a Data flow diagram.' Explain this sentence with an appropriate diagrammatic example. (1 mark)
- f. We wish to collect information about researchers, their institutions and research interests and then design a database that will track their activities. Draw an ERD to model the relationships between identified entities. [Hint: Each researcher is associated with only one institution and each researcher has several research interests]. (3 marks)
- g. You are designing a hospital management system for a client who has given you a short time frame of just one month. Which system acquisition strategy do you choose? Enumerate any three factors that supports your choice. (1.5 marks)

BREAK IT DOWN

- ASKED** Seven short parts, a to g, spanning DFD (a, e), ERD notation and theory (b, c, d, f) and acquisition (g). Modules Three, Four and Six.
- WATCH** Part b i explicitly says Information Engineering style, so use the crow's foot. Part d's "zero or many" and "one or more" phrases give you the exact cardinalities to mark.

A. FOUR REASONS TO MODEL (2 MARKS)

A model is a clear picture non-technical users can check; it separates what the system does from how it is built; it shows the system boundary and everything it touches; and it breaks a large system into manageable processes.

B. CARDINALITY, OPTIONALITY, DATA BALANCING (5.5 MARKS)

i. In crow's foot: one-to-one (a bar at each end), one-to-many (a bar at one end, a crow's foot at the other), many-to-many (a crow's foot at each end); draw each on a line between two entity rectangles. **ii.** A relationship is optional when the minimum is zero, drawn with a **circle** on the line (a customer may have zero orders). **iii.** Data balancing means the data flowing into and out of a process on a parent diagram must match the data flowing into and out of its child diagram when it is exploded.

C. PROCESS MODELLING VERSUS DATA MODELLING (1.5 MARKS)

PROCESS MODELLING (DFD)	DATA MODELLING (ERD)
Shows data in motion	Shows data at rest
Built from processes and flows	Built from entities and relationships
Answers how data moves	Answers what data is stored

ANSWER, CONTINUED**D. SALES MANAGEMENT ERD (3 MARKS)**

Entities SALES REP, CUSTOMER, ORDER, PRODUCT, WAREHOUSE. SALES REP serves zero-or-many CUSTOMERs, each CUSTOMER served by one rep; CUSTOMER places one-or-many ORDERs, each ORDER by one customer; ORDER contains one-or-many PRODUCTs; WAREHOUSE stores zero-or-many PRODUCTs. Mark the optional (circle) and many (crow's foot) ends as the words say.

E. DATA CANNOT MOVE BETWEEN ENTITIES (1 MARK)

A data flow must touch a process at one end, so you cannot draw an arrow straight from one external entity rectangle to another: put a process bubble between them. For example, Customer to Bank is wrong; Customer to "1.0 Process payment" to Bank is right.

F. RESEARCHER ERD (3 MARKS)

Entities RESEARCHER, INSTITUTION, RESEARCH INTEREST. A RESEARCHER belongs to one INSTITUTION (many researchers per institution, so INSTITUTION 1:M RESEARCHER); a RESEARCHER has many RESEARCH INTERESTs (1:M, or M:N if interests are shared). Crow's foot with the one bar at the institution end.

G. ACQUISITION STRATEGY (1.5 MARKS)

With only one month, **buy a packaged solution or outsource**, not build. Factors: the very short time available, the lack of in-house build capacity in the time, and that a hospital admin system is not so unique that a package cannot fit.

- a. As the head of an interview panel to recruit system analyst for your company, explain in detail three (3) skills you will be looking for in the successful candidate. (7 marks)
- b. Draw the Class diagram showing the Relationships and Multiplicities for the following analysis information of hospital management system
- A Patient is made-up of one Patient Details (Patient details is PId, Name, Age, gender)
 - A Patient can either be a New Patient or an Old Patient.
 - A Ward can be accommodated by at least 8 Patients per time.
 - A Ward is made-up of one Ward Details (Ward details is WardNo., No_of_Bed, WType).
 - A Convenience Room is part of a Ward.
 - A Ward has at least two Nurses attached to it per time. (5 marks)
- c. Briefly describe the following Object-Oriented Design concepts with relevant examples: i) Polymorphism, ii) Inheritance, iii) Composition and iv) Aggregation (4.5 marks)

BREAK IT DOWN

ASKED The analyst's skills (a), a class diagram from a bulleted spec (b), and four object-oriented concepts (c). Modules One and Five.

WATCH Each bullet in b maps to a relationship: "made-up of one" is composition, "either or" is generalization, "at least 8" and "at least two" are multiplicities, "part of" is composition.

ANSWER

A. THREE SKILLS (7 MARKS)

Technical: understanding of computers, networks, databases and the tools of building, so the analyst can understand and communicate technical requirements. **Business:** understanding of how organisations work and what this business does, so the system fits real goals. **People (interpersonal):** the most important; communicating in interviews and reviews, managing expectations and resolving conflict, because the analyst depends on users, managers and programmers.

B. HOSPITAL CLASS DIAGRAM (5 MARKS)

Classes PATIENT, PATIENTDETAILS, NEWPATIENT, OLDPATIENT, WARD, WARDDETAILS, CONVENIENCEROOM, NURSE. PATIENT is composed of one PATIENTDETAILS (filled diamond, 1); NEWPATIENT and OLDPATIENT generalise to PATIENT (hollow triangle); WARD holds 8..* PATIENTs; WARD is composed of one WARDDETAILS; CONVENIENCEROOM is part of WARD (filled diamond); WARD has 2..* NURSEs. Draw it exactly as the worked example in Module Five.

C. FOUR OOD CONCEPTS (4.5 MARKS)

Polymorphism: the same method behaves differently on different classes (draw() on a Circle versus a Square). **Inheritance:** a subclass takes on a superclass's members (SavingsAccount from Account). **Composition:** a life-or-death has-a (a House of Rooms). **Aggregation:** a loose has-a (a Team of Players).

- a. A System analyst is a problem solver. Explain your answer with tangible reasons. (5.5 marks)
- b. Patients call or come to the hospital to make new appointments as well as change or cancel appointments. Check to make sure the patient is a current patient and has no unpaid bills. If this is a new patient, perform "Add patient" before continuing. If the patient has unpaid bills, forward the call or patient to the account's office. If it is a change or cancellation, find the current appointment file and cancel it. Match the patient's desired appointment times with available dates and times and schedule new appointment.
 - i) Draw a Use Case Diagram for this scenario (6 marks)
 - ii) Draw an activity diagram for the scenario. (6 marks)

BREAK IT DOWN

ASKED Why the analyst is a problem solver (a), then a use case diagram and an activity diagram for the appointment process (b). Modules One, Two and Five. This appointment scenario is the same one the 18/19 and 22/23 papers used.

ANSWER

A. THE ANALYST AS PROBLEM SOLVER (5.5 MARKS)

The analyst studies a business to find where it loses time, money or accuracy, works out whether technology can help, and specifies the system that solves the problem. The solution is usually an information system, but not always. The analyst researches and understands the problem, verifies the benefit outweighs the cost, defines requirements, develops alternatives, recommends the best, details it, implements and monitors, which is exactly a problem-solving sequence. That is why the analyst is a business problem solver and the bridge between users and the technical team.

B. USE CASE AND ACTIVITY DIAGRAMS (12 MARKS)

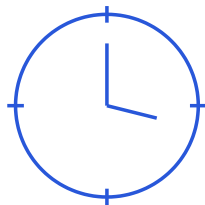
Use case (i): actor PATIENT; use cases Make appointment, Change appointment, Cancel appointment, Add patient. "Make appointment" **includes** "Check patient and bills". Draw it as Figure 2.1 in Module Two with these labels.

Activity (ii): start, "Receive call", decision "current patient?" (no leads to "Add patient" then continue), decision "unpaid bills?" (yes leads to "Forward to accounts" then end), then decision "change or cancel?": if so "Find and cancel appointment"; otherwise "Match desired times to available dates" and "Schedule appointment"; end. Draw it as Figure 5.3.

MOCK EXAMINATION ONE

Sit This Under Exam Conditions

Three hours, no notes. Answer one question in Section A and any three in Section B, exactly as in the hall. These questions are ours, built to the exact shape of the 2025/2026 paper with fresh scenarios, so you practise the method and not a memorised answer. Attempt the whole paper before you turn to the answers.



Question Paper

Time: three hours. Section A: answer one. Section B: answer any three. Marks in brackets. These are practice questions written for this manual, not a past paper.

Section A · Answer only one question.

PRACTICE QUESTION

MOCK 1 · Q1

An online bookstore lets a customer browse books, place an order, and pay. Each order is placed by exactly one customer, but a customer may place many orders over time. Each order contains one or more books, and a book may appear on many orders. A courier delivers each paid order to the customer.

- Draw an ERD to model the relationships between the identified entities. (4.5 marks)
- Data Flow Diagram (DFD): (i) Draw a context diagram for the entire system. (1.5 marks) (ii) Draw a Level 0 DFD showing the major processes. (4.5 marks) (iii) Draw a Level 1 DFD for the payment process. (3 marks)
- State three reasons the project could succeed and three reasons it could fail. (3 marks)
- Read and interpret, using forward and backward chaining, a relationship in which a BOOK is joined to an ORDER with a crow's foot at each end. (1 mark)

PRACTICE QUESTION

MOCK 1 · Q2

A school library system lets a member borrow and return books. A member must be a registered student or a member of staff before borrowing. A librarian records each loan. A member can hold many loans, but each loan belongs to one member.

- Draw a class diagram for this scenario, showing multiplicity where necessary. (5 marks)
- Draw a use case diagram for the system. (5 marks)
- Draw an activity diagram for the borrowing process. (5 marks)
- Differentiate between an object and a class with an example. (2.5 marks)

Section B · Answer any three questions.

PRACTICE QUESTION**MOCK 1 · Q3**

A neighbourhood laundry wants to automate its booking and collection business. They hire an analyst, Aisha, to guide them.

- a. Explain the three questions the team must answer during the feasibility analysis phase. (3 marks)
- b. Aisha's first step is to research and understand the problem. Outline six other steps she can take. (3 marks)
- c. How should Aisha explain to the owners what a model is? (4 marks)
- d. Besides people skills, explain one other skill Aisha needs to do a great job. (3 marks)
- e. What three activities will the support group participate in after the system is live? (3 marks)
- f. Mention three other job roles Aisha could fill. (1.5 marks)

PRACTICE QUESTION**MOCK 1 · Q4**

A logistics company wants to acquire a fleet-tracking system quickly; it has no strategic software skills in-house.

- a. Which system acquisition strategy fits, and what three factors support your choice? (3 marks)
- b. Name two software architectural functions the system will need. (3 marks)
- c. Besides operational requirements, name two other types of hardware non-functional requirement. (3 marks)
- d. Outline three principles of interface design. (3 marks)
- e. Design and populate a relational database of Driver, Trip and Payment, with primary keys, at least three columns and five rows each. (5.5 marks)

PRACTICE QUESTION**MOCK 1 · Q5**

A state university wants to replace its manual, paper-based student result processing with an integrated results and transcript system. Lecturers want speed, students want accuracy and quick access, and the registry insists on strict audit trails and reliability. The system must be delivered within one academic session.

- a. Briefly compare seven methodologies that could be used, saying why each is suitable or not. (7 marks)
- b. Identify the five activities in project planning. (5 marks)
- c. From the case, formulate at least: two business requirements, two user requirements, three functional requirements, two non-functional requirements, and two system requirements. (5.5 marks)

A hospital group wants to build a telemedicine platform so patients can consult doctors by video, share records, and receive e-prescriptions. At the start the requirements are unclear, and halfway through, management asks to add online payment and pharmacy delivery.

- a. Identify the weaknesses of the as-is system, the improvements in the to-be system, and the assumptions to validate before development. (3.5 marks)
- b. Design a requirements elicitation plan using at least four techniques; for each, give the stakeholders, the information collected, and why it suits this project. (4 marks)
- c. Write short notes on UML, stating its inventors and uses. (2 marks)
- d. Categorise UML diagrams into two broad categories, giving two diagrams and their function in each. (4 marks)
- e. With diagrams, explain inheritance, composition, aggregation and association. (4 marks)

Answers

Mark your own work honestly. Where an answer says "draw", check your diagram against the figure named in the relevant module.

ANSWER

MOCK 1 · Q1

- a. ERD.** Entities: CUSTOMER, ORDER, BOOK, COURIER. CUSTOMER 1:M ORDER (a customer places many orders, each order one customer); ORDER M:N BOOK (an order has many books, a book on many orders, resolved with an ORDER-LINE linking entity); COURIER 1:M ORDER (a courier delivers many orders). Crow's foot throughout.
- b. DFDs.** Context: one process "0 Bookstore" between CUSTOMER and COURIER, with order and payment in, confirmation and delivery out. Level 0: 1.0 Place order, 2.0 Take payment, 3.0 Arrange delivery, plus stores D1 Orders and D2 Books. Level 1 of 2.0: 2.1 Validate card, 2.2 Charge, 2.3 Issue receipt, balancing 2.0.
- c.** Succeed: clear requirements, user involvement, realistic schedule. Fail: unclear requirements, no user involvement, weak management support.
- d.** A crow's foot at each end is many-to-many: forward, one BOOK appears on many ORDERS; backward, one ORDER contains many BOOKS.

ANSWER

MOCK 1 · Q2

- a. Class diagram.** Classes MEMBER, STUDENT, STAFF, LOAN, BOOK, LIBRARIAN. STUDENT and STAFF generalise to MEMBER (hollow triangle). MEMBER 1 to * LOAN; LOAN * to 1 BOOK; LIBRARIAN records * LOAN. Give each class two attributes and a method.
- b. Use case.** Actors MEMBER and LIBRARIAN; use cases Register, Borrow book, Return book, Record loan. "Borrow book" includes "Check membership".
- c. Activity.** Start, "Request book", decision "registered member?" (no ends, yes continues), "Check availability", "Record loan", "Issue book", end.
- d.** A class is a blueprint (Book); an object is an instance of it (the specific copy of "Systems Analysis" with barcode 0012).

- a.** Is it technically feasible (can we build it)? Economically feasible (worth the cost)? Operationally feasible (will it be used and fit the business)?
- b.** Verify benefit outweighs cost; define requirements; develop alternatives; recommend the best; detail the chosen solution; implement (then monitor).
- c.** A model is a simplified picture of the system that shows the important parts so everyone can agree them before building, like an architect's drawing.
- d.** Technical skills: understanding computers, networks and databases and the tools of building, so she can specify and communicate technical requirements. (Business knowledge is also acceptable.)
- e.** Maintain the system, enhance the system, support the users (help desk).
- f.** Project manager, software architect, product designer.

- a.** Buy a packaged solution or outsource, not build; factors: short time available, lack of in-house skills, the system is not strategic (also cost, package fit).
- b.** Any two of data storage, data access logic, application logic, presentation logic.
- c.** Performance and security (the fourth is cultural/political).
- d.** Consistency, simplicity/clarity, user feedback.
- e.** Three tables, each with a primary key: Driver(driverId PK, name, phone), Trip(tripId PK, driverId FK, distance), Payment(paymentId PK, tripId FK, amount), five dummy rows each; driverId and tripId are the linking foreign keys.

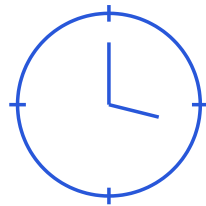
- a.** Compare waterfall (strong documentation, rigid), prototyping (good for unclear needs, can be mistaken for finished), RAD (fast, weak on large systems), spiral (risk analysis each cycle, heavy), incremental (early delivery, integration effort), agile (handles change, light documentation clashes with audit needs), V-shape (test pairing, reliable, rigid). For a results system needing audit trails and one-session delivery, an incremental or V-shape approach fits well.
- b.** Define the problem, produce the schedule, confirm feasibility, staff the project, launch it.
- c.** Business: cut result-processing time, guarantee an audit trail. User: a lecturer enters marks online, a student views results instantly. Functional: record marks, compute the grade, generate a transcript. Non-functional: 99.9% availability, encrypt results. System: store results as time-stamped records keyed by matriculation number, integrate with the student database.

- a.** As-is: no remote consultation, records on paper, no e-prescriptions. To-be: video consultation, shared records, e-prescriptions. Assumptions: patients have devices and connectivity; the platform can integrate with hospital records; video quality is adequate for diagnosis.
- b.** Interview doctors (clinical workflow; depth); questionnaire to patients (what they want; reaches many); observe a clinic (real process); review existing records (data and integration).
- c.** UML is a standard notation for modelling object-oriented systems, created by Booch, Rumbaugh and Jacobson; used to visualise, specify, construct and document a system.
- d.** Structural (class, object) model what the system is; behavioural (use case, activity, sequence) model what it does.
- e.** Inheritance (hollow triangle, is-a); composition (filled diamond, part dies with the whole); aggregation (hollow diamond, part survives); association (plain line).

MOCK EXAMINATION TWO

Sit This One After the First

Same rules, new scenarios. Three hours, one from Section A and any three from Section B. By now the routine should be automatic: read for nouns and verbs, draw boxes then lines then symbols, answer the verb in the instruction.



Question Paper

Time: three hours. Section A: answer one. Section B: answer any three. Practice questions written for this manual.

Section A · Answer only one question.

PRACTICE QUESTION

MOCK 2 · Q1

A hotel reservation system lets a guest book a room. Each booking is made by one guest, but a guest may make many bookings. Each booking is for one room, and a room may have many bookings over time. A receptionist confirms each booking and takes payment.

- Draw an ERD for the identified entities. (4.5 marks)
- DFD: (i) context diagram (1.5 marks); (ii) Level 0 (4.5 marks); (iii) Level 1 for the payment process (3 marks).
- Give three reasons the project could succeed and three it could fail. (3 marks)
- Interpret, forward and backward, a GUEST joined to a BOOKING with one at the guest end and many at the booking end. (1 mark)

PRACTICE QUESTION

MOCK 2 · Q2

A ride-hailing app connects a rider with a driver. A rider requests a trip; the system assigns one driver; each trip has one rider and one driver. A rider can be a standard rider or a premium rider. Payment is taken at the end of each trip.

- Draw a class diagram, showing multiplicity. (5 marks)
- Draw a use case diagram. (5 marks)
- Draw an activity diagram for requesting and completing a trip. (5 marks)
- State three drawbacks of multiple inheritance. (2.5 marks)

Section B · Answer any three questions.

PRACTICE QUESTION**MOCK 2 · Q3**

A bakery chain wants to automate ordering and delivery across its branches. They hire an analyst, Tunde, to lead the effort.

- a. Explain the three feasibility questions the team must answer. (3 marks)
- b. After researching and understanding the problem, outline six other steps Tunde can take. (3 marks)
- c. How should Tunde explain what a model is to the owners? (4 marks)
- d. Besides people skills, explain one other skill Tunde needs. (3 marks)
- e. What three activities will the support group participate in? (3 marks)
- f. Mention three other job roles Tunde could fill. (1.5 marks)

PRACTICE QUESTION**MOCK 2 · Q4**

A community bank wants to acquire a loan-management system fast; it lacks strategic software skills.

- a. Which acquisition strategy fits, and three supporting factors? (3 marks)
- b. Name two software architectural functions the system needs. (3 marks)
- c. Besides operational requirements, name two other hardware non-functional requirements. (3 marks)
- d. Outline three interface design principles. (3 marks)
- e. Design and populate a relational database of Account, Loan and Payment, with primary keys, at least three columns and five rows each. (5.5 marks)

PRACTICE QUESTION**MOCK 2 · Q5**

A courier company wants to replace its manual parcel-tracking with an integrated tracking and delivery system. Dispatchers want speed, customers want accurate live tracking, and management insists on reliability and audit trails. Delivery is required within nine months.

- a. Compare seven methodologies, saying why each suits this project or not. (7 marks)
- b. Identify the five project planning activities. (5 marks)
- c. Formulate at least two business, two user, three functional, two non-functional and two system requirements from the case. (5.5 marks)

A polytechnic wants to build an e-learning platform for lectures, assignments and grading. Requirements are unclear at first; midway, management asks to add live classes and a fee-payment portal.

- a. Identify the as-is weaknesses, to-be improvements, and assumptions to validate. (3.5 marks)
- b. Design an elicitation plan with at least four techniques, each with stakeholders, information and justification. (4 marks)
- c. Write short notes on UML with its inventors and uses. (2 marks)
- d. Categorise UML diagrams into two categories with two examples and their function each. (4 marks)
- e. With diagrams, explain inheritance, composition, aggregation and association. (4 marks)

Answers

Check each "draw" answer against the figure named in the relevant module.

ANSWER

MOCK 2 · Q1

- a.** Entities GUEST, BOOKING, ROOM, RECEPTIONIST. GUEST 1:M BOOKING; ROOM 1:M BOOKING; RECEPTIONIST 1:M BOOKING. Crow's foot throughout.
- b.** Context: one process "0 Reservation" between GUEST and RECEPTIONIST. Level 0: 1.0 Make booking, 2.0 Take payment, 3.0 Confirm, with stores D1 Bookings, D2 Rooms. Level 1 of 2.0: validate, charge, receipt, balanced.
- c.** Succeed: clear requirements, user involvement, realistic schedule. Fail: unclear requirements, no user involvement, weak management support.
- d.** One at the guest end, many at booking: forward, one GUEST makes many BOOKINGS; backward, one BOOKING is made by one GUEST.

ANSWER

MOCK 2 · Q2

- a.** Classes RIDER, STANDARDRIDER, PREMIUMRIDER, DRIVER, TRIP, PAYMENT. STANDARDRIDER and PREMIUMRIDER generalise to RIDER (hollow triangle). RIDER 1 to * TRIP; DRIVER 1 to * TRIP; TRIP 1 to 1 PAYMENT.
- b.** Actors RIDER, DRIVER; use cases Request trip, Assign driver, Complete trip, Pay. "Complete trip" includes "Take payment".
- c.** Start, "Request trip", decision "driver available?" (no ends, yes continues), "Assign driver", "Make trip", "Take payment", end.
- d.** The diamond problem (ambiguity), increased complexity, name clashes.

ANSWER

MOCK 2 · Q3

- a.** Technically feasible? Economically feasible? Operationally feasible?
- b.** Verify benefit outweighs cost; define requirements; develop alternatives; recommend the best; detail the solution; implement (then monitor).
- c.** A model is a simplified picture that shows the important parts so all can agree them before building, like an architect's drawing.
- d.** Technical or business knowledge and skills, with an explanation of how it helps him specify the system.
- e.** Maintain, enhance, support the users (help desk).
- f.** Project manager, software architect, product designer.

- a.** Buy a package or outsource; factors: short time, no in-house skills, not strategic.
- b.** Any two of data storage, data access logic, application logic, presentation logic.
- c.** Performance and security (fourth: cultural/political).
- d.** Consistency, simplicity/clarity, user feedback.
- e.** Account(accId PK, holder, balance), Loan(loanId PK, accId FK, amount), Payment(payId PK, loanId FK, amount); five dummy rows each; accId and loanId are the foreign keys.

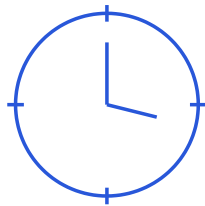
- a.** As in the manual: waterfall (documented, rigid), prototyping (unclear needs, mistaken for finished), RAD (fast, weak on large), spiral (risk analysis, heavy), incremental (early value, integration), agile (handles change, light docs), V-shape (test pairing, reliable, rigid). A tracking system needing audit trails and nine-month delivery suits incremental or V-shape.
- b.** Define the problem, schedule, confirm feasibility, staff, launch.
- c.** Business: faster delivery, full audit trail. User: a dispatcher assigns a parcel, a customer tracks live. Functional: register parcel, update status, notify customer. Non-functional: high availability, encrypt data. System: store status as time-stamped events keyed by parcel ID, integrate with the customer database.

- a.** As-is: no online lectures, manual grading, no analytics. To-be: online lectures, assignments, automated grading. Assumptions: students have connectivity, the platform integrates with existing records, the fee portal is available.
- b.** Interview lecturers (requirements, depth), questionnaire to students (needs, reach), observe a class (real use), review the existing system (data, integration).
- c.** UML: a standard object-oriented modelling notation by Booch, Rumbaugh and Jacobson; used to visualise, specify, construct and document.
- d.** Structural (class, object) and behavioural (use case, activity/sequence).
- e.** Inheritance (triangle, is-a), composition (filled diamond), aggregation (hollow diamond), association (plain line).

MOCK EXAMINATION THREE

One More, To Be Sure

The last rehearsal. If you can answer this paper cleanly, in time, with every diagram drawn without hesitation, you are ready for the hall. Three hours, one from Section A and any three from Section B.



Question Paper

Time: three hours. Section A: answer one. Section B: answer any three. Practice questions written for this manual.

Section A · Answer only one question.

PRACTICE QUESTION

MOCK 3 · Q1

A car rental system lets a customer rent a car. Each rental is by one customer, but a customer may have many rentals. Each rental is for one car, and a car may be rented many times. A staff member processes each rental and takes payment.

- Draw an ERD for the identified entities. (4.5 marks)
- DFD: (i) context diagram (1.5 marks); (ii) Level 0 (4.5 marks); (iii) Level 1 for the payment process (3 marks).
- Give three reasons the project could succeed and three it could fail. (3 marks)
- Interpret, forward and backward, a CUSTOMER joined to a RENTAL with one at the customer end and many at the rental end. (1 mark)

PRACTICE QUESTION

MOCK 3 · Q2

A hospital pharmacy system dispenses drugs against prescriptions. A pharmacist dispenses each prescription; a prescription belongs to one patient; a patient may have many prescriptions. A prescription can be an inpatient prescription or an outpatient prescription.

- Draw a class diagram, showing multiplicity. (5 marks)
- Draw a use case diagram. (5 marks)
- Draw an activity diagram for dispensing a prescription. (5 marks)
- Differentiate between an object and a class with an example. (2.5 marks)

Section B · Answer any three questions.

PRACTICE QUESTION**MOCK 3 · Q3**

A tailoring business wants to automate measurements, orders and delivery. They hire an analyst, Ngozi, to guide them.

- a. Explain the three feasibility questions the team must answer. (3 marks)
- b. After researching and understanding the problem, outline six other steps Ngozi can take. (3 marks)
- c. How should Ngozi explain what a model is to the owners? (4 marks)
- d. Besides people skills, explain one other skill Ngozi needs. (3 marks)
- e. What three activities will the support group participate in? (3 marks)
- f. Mention three other job roles Ngozi could fill. (1.5 marks)

PRACTICE QUESTION**MOCK 3 · Q4**

A supermarket wants to acquire a point-of-sale system quickly; it has no strategic software skills.

- a. Which acquisition strategy fits, and three supporting factors? (3 marks)
- b. Name two software architectural functions the system needs. (3 marks)
- c. Besides operational requirements, name two other hardware non-functional requirements. (3 marks)
- d. Outline three interface design principles. (3 marks)
- e. Design and populate a relational database of Product, Sale and Payment, with primary keys, at least three columns and five rows each. (5.5 marks)

PRACTICE QUESTION**MOCK 3 · Q5**

A government agency wants to replace its manual, paper-based tax filing with an integrated online tax system. Citizens want ease of use, the agency wants accuracy and strict audit trails, and security is critical. Delivery is required within one year.

- a. Compare seven methodologies, saying why each suits this project or not. (7 marks)
- b. Identify the five project planning activities. (5 marks)
- c. Formulate at least two business, two user, three functional, two non-functional and two system requirements from the case. (5.5 marks)

A university wants to build a smart-campus platform linking attendance, library and cafeteria services. Requirements are unclear at first; midway, management asks to add access control and mobile payments.

- a. Identify the as-is weaknesses, to-be improvements, and assumptions to validate. (3.5 marks)
- b. Design an elicitation plan with at least four techniques, each with stakeholders, information and justification. (4 marks)
- c. Write short notes on UML with its inventors and uses. (2 marks)
- d. Categorise UML diagrams into two categories with two examples and their function each. (4 marks)
- e. With diagrams, explain inheritance, composition, aggregation and association. (4 marks)

Answers

Check each "draw" answer against the figure named in the relevant module. If any answer here surprises you, go back to the module it comes from before the exam.

ANSWER

MOCK 3 · Q1

- a.** Entities CUSTOMER, RENTAL, CAR, STAFF. CUSTOMER 1:M RENTAL; CAR 1:M RENTAL; STAFF 1:M RENTAL. Crow's foot throughout.
- b.** Context: one process "0 Rental" between CUSTOMER and STAFF. Level 0: 1.0 Make rental, 2.0 Take payment, 3.0 Return car, stores D1 Rentals, D2 Cars. Level 1 of 2.0: validate, charge, receipt, balanced.
- c.** Succeed: clear requirements, user involvement, realistic schedule. Fail: unclear requirements, no user involvement, weak management support.
- d.** One customer, many rentals: forward, one CUSTOMER has many RENTALS; backward, one RENTAL belongs to one CUSTOMER.

ANSWER

MOCK 3 · Q2

- a.** Classes PATIENT, PRESCRIPTION, INPATIENTPRESCRIPTION, OUTPATIENTPRESCRIPTION, PHARMACIST, DRUG. INPATIENT and OUTPATIENT generalise to PRESCRIPTION (hollow triangle). PATIENT 1 to * PRESCRIPTION; PHARMACIST dispenses * PRESCRIPTION; PRESCRIPTION * to * DRUG (via a line item).
- b.** Actors PATIENT, PHARMACIST; use cases Present prescription, Dispense drug, Record. "Dispense" includes "Check stock".
- c.** Start, "Receive prescription", decision "in stock?" (no ends, yes continues), "Dispense", "Record", end.
- d.** A class is a blueprint (Drug); an object is an instance (the specific batch of Paracetamol with batch number B21).

ANSWER

MOCK 3 · Q3

- a.** Technically feasible? Economically feasible? Operationally feasible?
- b.** Verify benefit outweighs cost; define requirements; develop alternatives; recommend the best; detail the solution; implement (then monitor).
- c.** A model is a simplified picture that shows the important parts so all can agree them before building, like an architect's drawing.
- d.** Technical or business knowledge and skills, explained.
- e.** Maintain, enhance, support the users (help desk).
- f.** Project manager, software architect, product designer.

- a.** Buy a package or outsource; factors: short time, no in-house skills, not strategic.
- b.** Any two of data storage, data access logic, application logic, presentation logic.
- c.** Performance and security (fourth: cultural/political).
- d.** Consistency, simplicity/clarity, user feedback.
- e.** Product(prodId PK, name, price), Sale(saleId PK, prodId FK, qty), Payment(payId PK, saleId FK, amount); five dummy rows each; prodId and saleId are the foreign keys.

- a.** Waterfall (documented, rigid), prototyping (unclear needs, mistaken for finished), RAD (fast, weak on large), spiral (risk analysis, heavy), incremental (early value, integration), agile (handles change, light docs), V-shape (test pairing, reliable, rigid). A tax system needing security and audit trails suits V-shape or a disciplined incremental approach.
- b.** Define the problem, schedule, confirm feasibility, staff, launch.
- c.** Business: cut filing time, ensure audit trails. User: a citizen files online, an officer reviews a return. Functional: accept a return, compute tax due, issue a receipt. Non-functional: high availability, strong encryption. System: store returns as time-stamped records keyed by tax ID, integrate with the payment gateway.

- a.** As-is: separate, manual campus services, no single view, no analytics. To-be: one platform linking attendance, library and cafeteria. Assumptions: students carry compatible cards or phones, services can integrate, the network covers campus.
- b.** Interview service heads (rules, depth), questionnaire to students (needs, reach), observe a service point (real use), review existing systems (data, integration).
- c.** UML: a standard object-oriented modelling notation by Booch, Rumbaugh and Jacobson; used to visualise, specify, construct and document.
- d.** Structural (class, object) and behavioural (use case, activity/sequence).
- e.** Inheritance (triangle, is-a), composition (filled diamond), aggregation (hollow diamond), association (plain line).